Guide complet d'installation

SQL Server 2022 – Always On Availability Groups

Domaine optimedit.eu — SQLS01 / SQLS02 — Réseau 192.168.1.0/24 — Intégration IIS (13 applications)

Préparé pour : Driss

Version 2.10— gestion des secrets et cohérence documentaire

Sommaire

Sommaire 2
0. Changelog V2.10 — correctifs de revue technique 5
1. Vue d'ensemble et architecture 7
1.1 Schéma d'architecture 7
1.2 Composants clés 7
2. Bonne pratique : ordre des opérations 8
2.1 Séquence recommandée 8
2.2 Vérification préalable de l'infrastructure (pré-vol) 8
3. Phase 1 — Préparation des serveurs SQL 10
3.0 Gestion des secrets 10
3.1 Structure de dossiers (sur S:\SQL) 11
3.2 Comptes de service AD 11
Explication du mécanisme des comptes de service SQL Server 13
3.3 Installation de SQL Server 2022 14
3.3.1 Vérification du prérequis : 15
3.3.2 Points de vigilance avant exécution 15
3.3.3 Configuration du certificat SQL server 17
3.3.3.1 Ajouter Administrateur aux sysadmins SQL Server 18
3.3.3.2 Création d'un modèle de certificat auto-enrôlement 19
3.3.3.3 Attribution des permissions sur la clé privée 21
3.3.3.4 Sélection du certificat dans SQL Server Configuration Manager 21
3.3.3.5 Vérification de la configuration 22
3.3.3.6 Application et redémarrage du certificat 23
3.3.4 Procédure de vérification après l'installation 23
3.3.4.1 Vérifier que les services sont démarrés sur SQLS01 et SQLS02 23
3.3.4.2 Vérifier la version de SQL Server 23
3.3.4.3 Vérifier les emplacements des fichiers sur SQLS01 et SQLS02 23
3.3.4.4 Vérifier les logins SQL Server SQLS01 et SQLS02 24
3.3.4.5 Vérifier les rôles serveur SQLS01 et SQLS02 24
3.4 Configuration réseau 25
3.5 Récapitulatifs (pré-vol & phase 1) 27
4. Phase 2 — Cluster Windows 28
Prérequis à vérifier avant de commencer 29
4.1 Installation de la fonctionnalité 29
4.2 Validation du cluster 30
4.3 Création du cluster 32
4.3.1 Création du partage ClusterWitness (sur OPT-FS03) 32
4.4 Quorum 34
Vérification : Le Quorum est-il Bien Fonctionnel ? 35
5. Phase 3 — Always On SQL Server 40
5.0.0 Creation de SPN 40

5.0.1 Verification si authentication Kerberos ou NTLM 41
5.0.2 Activation des logs Audit Kerberos 42
5.1 Activer Always On 43
5.2 Créer les endpoints Always On 43
5.3 Créer l'Availability Group (sur SQLS01, primaire) 43
5.4 Rejoindre l'AG (sur SQLS02, secondaire) 44
5.5 Créer le Listener 44
5.6 Tester le failover 44
6. Phase 4 — Création des bases de données 45
6.1 Création des 13 bases (sur SQLS01) 45
6.2 Ajout des bases à l'AG 45
7. Phase 5 — Sécurisation et maintenance 46
7.1 Transparent Data Encryption (TDE) 46
7.1.1 Restaurer le certificat TDE sur le secondaire 46
7.2 Jobs SQL Server Agent (Primary) 46
7.2.1 Synchroniser les jobs sur le secondaire 46
7.3 Alertes SQL Server 47
7.4 Audits SQL Server 47
7.5 Optimisation TempDB 47
8. Phase 6 — Intégration IIS (sessions SQL Server) 48
8.1 Comptes de service applicatifs (13 comptes svc_*) 48
8.2 Groupe Active Directory GG_IIS_AppPools 48
8.3 Base de sessions ASPState (avant le login — ordre corrigé) 48
8.4 Login SQL Server (après ASPState) 48
8.5 Configuration des 13 sites IIS 48
8.6 Tests de validation 48
9. Tâches d'administration récurrentes 50
9.1 Quotidien 50
9.2 Hebdomadaire 50
9.3 Mensuel 50
9.4 Sauvegardes de clés AES vers un serveur de fichier 50
9.4.1 Création du partage SQLSecrets$ 50
9.4.2 Sauvegarde des secrets vers OPT-FS03 50
9.5 Sauvegarde des clés AES dans KeePass 51
9.5.1 Procédure de sauvegarde 51
9.5.2 Procédure de restauration 51
9.5.3 Avertissements de sécurité 52
10. Annexe — Choix de la collation 52
11. Supervision avancée d'Always On 53
11.1 DMV essentielles 53
11.2 Compteurs Perfmon 53
11.3 Extended Events 53

11.4 Health Detection 53
12. Sauvegardes de bases dans un Availability Group 53
12.1 Préférence de sauvegarde 53
12.2 Job de sauvegarde AG-aware 53
12.3 Stratégie de sauvegarde complete 53
12.3.1 Sauvegarde SQL (jobs AG-aware) — déjà couvert 54
12.3.2 Sauvegarde VM-level (Veeam Backup & Replication) 54
12.3.3 Comparison des deux approaches 55
12.3.4 Recommandations pour le PoC et la production 56
13. Checklist complète des tests de bascule 56
15. Gestion des mises à jour SQL Server 57
16. Annexe — Liste des scripts fournis 58
17. Prochaines étapes possibles 60

0. Changelog V2.10 — correctifs de revue technique

V2.1 — Correctifs de revue technique
- Certificat TDE restauré sur SQLS02 (script 7.1.1)
- Ordre ASPState / Login corrigé (8.3 avant 8.4)
- Sous-dossier Audit pré-créé (3.1.0)
- Jobs SQL Agent synchronisés sur SQLS02 (script 7.2.1)
- Démarrage automatique du service SQL Agent (/AGTSVCSTARTUPTYPE)

V2.2 — Gestion des secrets
- Nouvelle section 3.0 et script 3.0.0.manage_secrets.ps1
- Conversion de 3.3.0, 7.1.0, 7.1.1 pour lecture des secrets .enc
- Nouvelle section 9.4 et script 9.4.2.backup_secrets_to_fileserver.ps1
- Nouvelle section 9.5 (sauvegarde KeePass)

V2.3 — Cohérence documentaire
- Script 0.0.0.preflight_check.ps1 documenté en 2.2 et ajouté à l'annexe 16
- Renommage 7.1.0 et 7.1.1 en .ps1 (contenu PowerShell)
- Correction du titre 9.4 (extension dupliquée)
- Décompte des scripts corrigé (33 fichiers)
- Chapitre 17 réécrit (retrait d'Azure Key Vault, désormais couvert par 3.0)

V2.4 — Cohérence documentaire
- Extraits script corrigés dans ces trois sections : 7.1, 7.11, 3.3.

V2.5 — Cohérence documentaire
- Corriger 9 coquilles qui peuvent impacter l’exécution.

V2.6 — Ajout des scripts de création des partages OPT-FS03
- Nouveau script 4.3.1.create_cluster_witness_share.ps1 (partage ClusterWitness)
- Nouveau script 9.4.1.create_sqlsecrets_share.ps1 (partage SQLSecrets$)
- Renommage 9.4.0.backup_secrets_to_fileserver.ps1 → 9.4.2.backup_secrets_to_fileserver.ps1
- Section 9.4 restructurée en deux étapes : 9.4.1 (prérequis) et 9.4.2 (action)
- Décompte des scripts : 33 → 35
- Annexe 16 mise à jour

V2.7 — Cohérence documentaire
- Section 4.4 : remplacer D:\clusterwitness\ par P:\ClusterWitness\.
- Section 9.4.1 : remplacer D:\SQLSecrets\ par P:\SQLSecrets\.

V2.8 — Stratégie de sauvegarde complète
- Nouvelle section 12.3 — Stratégie de sauvegarde complète
12.3.1 — Sauvegarde SQL (jobs AG-aware) — rappel du chapitre 12.1/12.2
12.3.2 — Sauvegarde VM-level (Veeam Backup & Replication)
12.3.3 — Comparaison des deux approches
12.3.4 — Recommandations pour le PoC et la production
12.3.5 — Avertissement sur les checkpoints Hyper-V
- Ajout de la configuration Veeam Backup & Replication Community Edition
- Ajout de la recommandation sur le volume de sauvegarde dédié (distinct du disque S:\)
- Ajout de l'avertissement Microsoft sur l'usage des checkpoints Hyper-V sur les nœuds de cluster WSFC + Always On
- Aucune renumérotation : les chapitres 13 à 17 restent inchangés

V2.9 — Rajout de grande chapitres (Configuration du certificat SQL server + Procédure de vérification après l'installation)
3.3.3 Configuration du certificat SQL server
3.3.3.1 Ajouter Administrateur aux sysadmins SQL Server
3.3.3.2 Création d'un modèle de certificat auto-enrôlement
3.3.3.3 Attribution des permissions sur la clé privée

3.3.3.4 Sélection du certificat dans SQL Server Configuration Manager
3.3.3.5 Vérification de la configuration
3.3.3.6 Application et redémarrage du certificat

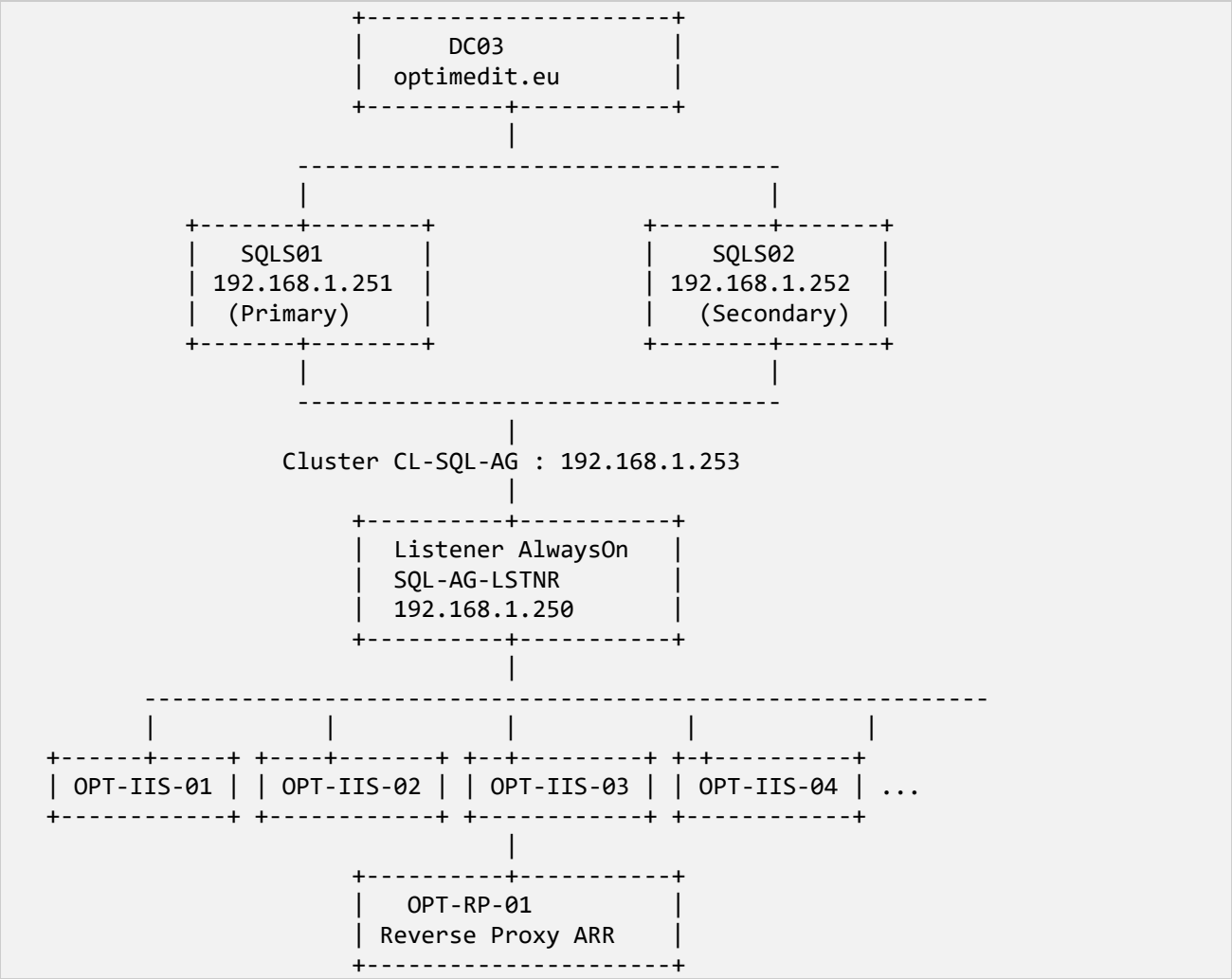
3.3.4 Procédure de vérification après l'installation
3.3.4.1 Vérifier que les services sont démarrés sur SQLS01 et SQLS02
3.3.4.2 Vérifier la version de SQL Server
3.3.4.3 Vérifier les emplacements des fichiers sur SQLS01 et SQLS02
3.3.4.4 Vérifier les logins SQL Server SQLS01 et SQLS02
3.3.4.5 Vérifier les rôles serveur SQLS01 et SQLS02

V2.10 — Correctifs de revue technique et enrichissement documentaire
Phase 2 — Cluster Windows : enrichissements
Nouveau chapitre "Prérequis à vérifier avant de commencer" dans la section 4 (Phase 2 — Cluster Windows) :
Section 4.2 — Validation du cluster : ajout de tableaux de validation
Section 4.3.1 — Création du partage ClusterWitness (sur OPT-FS03) : ajout d'une sous-section "Vérification Avant de passer à l'étape 4.4"
Section 4.4 — Quorum : ajout de trois nouvelles sous-sections
"Vérification : Le Quorum est-il Bien Fonctionnel ?" — 4 tests :
"Résolution de l'Avertissement Restant (AD Permissions)" — procédure détaillée de délégation de contrôle sur l'OU SQL Server :
"Vérification Post-Délégation (sur le DC03)" :
Vérification des ACE sur l'OU via PowerShell (Get-Acl)
Test définitif : vérification que le CNO est bien authentifié dans AD et que le nom réseau du cluster est en ligne
Phase 3 — Always On SQL Server : nouveau chapitre
Nouveau chapitre 5.0.0 — Création de SPN avec script 5.0.0.manage_sql_spn.ps1
Création des 8 SPN MSSQLSvc/* sur le compte de service SVC_SQL_Engine
SPN pour chaque nœud SQL (SQLS01, SQLS02) avec FQDN et nom court, avec et sans port
Nouvelle sous-section "Vérification si authentication Kerberos ou NTLM"

1. Vue d'ensemble et architecture

Ce guide décrit, dans l'ordre recommandé par Microsoft, l'ensemble des étapes pour déployer une architecture SQL Server 2022 Always On à deux nœuds, intégrée à 13 applications IIS via des sessions externalisées (SQL Server Session State), dans le domaine optimedit.eu, sur le réseau 192.168.1.0/24.

1.1 Schéma d'architecture



1.2 Composants clés

Composant	Valeur retenue
Nœud SQLS01 (Primary)	192.168.1.251
Nœud SQLS02 (Secondary)	192.168.1.252
IP du cluster (CL-SQL-AG)	192.168.1.253
IP du Listener (SQL-AG-LSTNR)	192.168.1.250
Édition SQL Server	SQL Server 2022, édition Developer (PoC – voir note ci-dessous)
Availability Group	AG-OPTIMEDIT
Collation	SQL_Latin1_General_CP1_CI_AS

Stockage dédié	S:\SQL\Data, S:\SQL\Log, S:\SQL\Backup (+ \Audit), S:\SQL\TempDB
Comptes de service SQL	SVC_SQL_Engine, SVC_SQL_Agent
Groupe AD applicatif	GG_IIS_AppPools (13 comptes svc_*)
Sessions IIS	SQL Server Session State (base ASPState) dans l'AG

Important : *Ce guide déploie une édition Developer, sous licence strictement limitée au développement, aux tests et aux démonstrations. Fonctionnellement identique à Enterprise (Always On y compris), elle ne doit pas être utilisée pour un service direct aux utilisateurs finaux. Le passage en production nécessite une licence Enterprise — l'architecture elle-même n'a pas à être refaite.*

Remarque : *Chaque étape de ce guide indique, dans un encadré vert, le script fourni qui l'automatise et le nœud sur lequel l'exécuter. L'ensemble des scripts est livré dans Scripts_SQLServer2022_AlwaysOn_optimedit.zip et récapitulé en annexe (chapitre 16).*

2. Bonne pratique : ordre des opérations

Règle d'or : toujours mettre en place le cluster, Always On et le Listener avant de créer les bases de données applicatives — et toujours créer ASPState avant d'attribuer les droits dessus.

2.1 Séquence recommandée

1. Installer SQL Server 2022 sur SQLS01 et SQLS02.
2. Créer le cluster Windows CL-SQL-AG (192.168.1.253).
3. Activer Always On et créer les endpoints sur les deux nœuds.
4. Créer l'Availability Group AG-OPTIMEDIT sur SQLS01, puis faire rejoindre SQLS02.
5. Créer le Listener SQL-AG-LSTNR (192.168.1.250).
6. Créer les 13 bases applicatives sur SQLS01, puis les ajouter à l'AG (seeding automatique).
7. Sécuriser (TDE + restauration du certificat sur SQLS02, jobs synchronisés sur les deux nœuds, alertes, audits, TempDB).
8. Créer la base ASPState et l'ajouter à l'AG.
9. Créer le groupe AD GG_IIS_AppPools, puis le login SQL (après ASPState, pas avant).
10. Configurer les 13 sites IIS pour utiliser le SQL Server Session State via le Listener.

2.2 Vérification préalable de l'infrastructure (pré-vol)

Avant de commencer la Phase 1, ce script vérifie **l'état des 5 serveurs cœur** (DC03, SQLS01, SQLS02, OPT-FS03, OPT-RP-01) — DNS, ping, WinRM, OS, RAM, CPU (cœurs physiques vs threads logiques), espace disque C:/S:, appartenance au domaine — avant toute installation. Il ne modifie rien : il remonte des alertes (RAM insuffisante, disque S: absent, CPU sous-dimensionné) à corriger avant de poursuivre.

Script fourni : `0.0.0.preflight_check.ps1` *— exécuté sur : poste d'administration (ou SQLS01)*

```
.\0.0.0.preflight_check.ps1 -IncludeIIS -ExportCsv "C:\Rapports\preflight.csv"
# Vérifie DNS, ping, WinRM, OS, RAM, CPU (cœurs physiques/logiques), disque C:/S:,
domaine, uptime
# pour DC03, SQLS01, SQLS02, OPT-FS03, OPT-RP-01 (+ les 6 IIS avec le paramètre -
IncludeIIS)
# Alerte si RAM < 8 Go, CPU < 4 cœurs physiques ou disque S: absent sur un nœud SQL
```

> **Remarque :** *aucun mot de passe n'est demandé (utilise les identifiants de la session) ; le script n'effectue aucune modification sur les serveurs cibles — il est sans risque à réexécuter à tout moment.*

3. Phase 1 — Préparation des serveurs SQL

3.0 Gestion des secrets

Script fourni : `3.0.0.manage_secrets.ps1` *— exécuté sur : SQLS01 + SQLS02*

Cette section met en place, **avant toute manipulation de mot de passe**, l'infrastructure de chiffrement des secrets utilisée par les scripts suivants. Une clé AES 256 bits est générée localement sur chaque nœud SQL (SQLS01 et SQLS02) et protégée par ACL strictes. Les mots de passe (comptes de service SQL, TDE, IIS) sont saisis interactivement puis chiffrés dans des fichiers **.enc** nommés explicitement, de sorte qu'aucun mot de passe en clair n'apparaisse jamais sur disque.

Le script *3.0.0.manage_secrets.ps1* automatise la génération de la clé, le chiffrement des secrets nommés et l'application des ACL. Il est **idempotent** : la clé AES n'est générée qu'une seule fois par nœud, et chaque secret est chiffré dans un fichier distinct (sql-service-pwd.enc, tde-masterkey-pwd.enc, etc.). Une note d'information en 3.0.1 renvoie vers la procédure de sauvegarde sécurisée des clés (section 9.5).

Usage	Nom du fichier `.enc`	Script consommateur
Mot de passe des comptes de service SQL (`SVC_SQL_Engine`, `SVC_SQL_Agent`)	`sql-service-pwd.enc`	`3.3.0` (installation SQL)
Mot de passe du master key TDE	`tde-masterkey-pwd.enc`	`7.1.0`
Mot de passe du backup certificat TDE	`tde-cert-pwd.enc`	`7.1.0`, `7.1.1`
Mot de passe des 13 comptes `svc_*` IIS	`iis-svc-pwd.enc`	*(aucun script pour l'instant — chiffré pour cohérence)*

> **Remarque** : Les scripts 3.2.0 et 8.1.0 saisissent le mot de passe directement via *Read-Host -AsSecureString* pour créer les comptes AD. Les fichiers *sql-service-pwd.enc* et *iis-svc-pwd.enc* sont produits séparément pour que ce même mot de passe puisse être consommé par d'autres scripts (par exemple 3.3.0 pour l'installation SQL Server). Si demain un nouveau secret apparaît (Database Mail, Linked Server, etc.), on ajoute simplement un nouveau fichier **.enc** avec un nom explicite. La logique reste la même : un usage, un fichier.

Exécution sur les deux nœuds :

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
.\3.0.0.manage_secrets.ps1 -GenerateKey


.\3.0.0.manage_secrets.ps1 -SecretName "sql-service-pwd"
```

Saisir le mot de passe des comptes de service SQL (celui qui sera utilisé pour **`SVC_SQL_Engine`** et **`SVC_SQL_Agent`**).

```
Get-ChildItem "C:\ProgramData\SQLSecrets" | Select-Object Name, Length,
LastWriteTime

Name                Length LastWriteTime
----                ------ -------------
aes.key                 32 23/09/2026 19:21:51
sql-service-pwd.enc    278 23/09/2026 19:24:11


Remarque : aes.key fait exactement 32 octets (256 bits). C'est la bonne taille.
```

3.1 Structure de dossiers (sur S:\SQL)

Dossier	Usage
S:\SQL\Data	Fichiers de données (.mdf / .ndf)
S:\SQL\Log	Fichiers de journal des transactions (.ldf)
S:\SQL\Backup	Sauvegardes FULL / DIFF / LOG
S:\SQL\Backup\Audit	Fichiers d'audit serveur (voir 7.4)
S:\SQL\TempDB	Fichiers TempDB

Script fourni : `3.1.0.create_storage_directories.ps1` — *exécuté sur : SQLS01 + SQLS02*

```
$RootPath = "S:\SQL"
foreach ($Folder in @("Data","Log","Backup","TempDB","Backup\Audit")) {
    $Path = Join-Path $RootPath $Folder
    if (!(Test-Path $Path)) { New-Item -Path $Path -ItemType Directory -Force | Out-Null
}
}
```

Exécution sur les deux nœuds :

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
.\3.1.0.create_storage_directories.ps1
```

```
Get-ChildItem -Path "S:\SQL" -Recurse -Directory | Select-Object FullName
```

3.2 Comptes de service AD

Créer dans AD les comptes SVC_SQL_Engine (moteur SQL Server) et SVC_SQL_Agent (SQL Server Agent), puis leur accorder Log on as a service et Perform volume maintenance tasks (instant file initialization).

Script fourni : `3.2.0.create_sql_service_accounts.ps1` — *exécuté sur : DC03*

```
$Accounts = @("SVC_SQL_Engine","SVC_SQL_Agent")
foreach ($Account in $Accounts) {
    if (-not (Get-ADUser -Filter "SamAccountName -eq '$Account'" -ErrorAction
SilentlyContinue)) {
        $Password = Read-Host -Prompt "Mot de passe pour $Account" -AsSecureString
        New-ADUser -Name $Account -SamAccountName $Account -AccountPassword $Password `
                   -PasswordNeverExpires $true -Enabled $true
    }
}
```

Exécution sur le DC03 :

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
.\3.2.0.create_sql_service_accounts.ps1
```

Ce script va demander :

- Le mot de passe pour SVC_SQL_Engine.
- Le mot de passe pour SVC_SQL_Agent.

Saisir exactement le même mot de passe que celui utilisé dans **`sql-service-pwd.enc`**

```
PS C:\Users\Administrateur> Set-Location "C:\Scripts\_Projet AlwaysOn"
PS C:\Scripts\_Projet AlwaysOn> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
PS C:\Scripts\_Projet AlwaysOn> .\3.2.0.create_sql_service_accounts.ps1
Mot de passe pour SVC_SQL_Engine: **********
Compte cree : SVC_SQL_Engine
Mot de passe pour SVC_SQL_Agent: **********
Compte cree : SVC_SQL_Agent
PS C:\Scripts\_Projet AlwaysOn> HOSTNAME
DC03
```

```
Get-ADUser -Filter "SamAccountName -like 'SVC_SQL_*'" | Select-Object Name, SamAccountName, Enabled
```

```
Name          SamAccountName Enabled
----          -------------- -------
SVC_SQL_Agent  SVC_SQL_Agent     True
SVC_SQL_Engine SVC_SQL_Engine    True
```

Script fourni : `3.2.1.grant_service_logon_rights.ps1` — *exécuté sur : SQLS01 + SQLS02*

```
# Accorde SeServiceLogonRight + SeManageVolumePrivilege
# aux comptes optimedit\SVC_SQL_Engine et optimedit\SVC_SQL_Agent
# via secedit (voir script complet dans l'archive)
```

Exécution sur les deux nœuds SQLS01 et SQLS02, en tant qu'administrateur :

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
.\3.2.1.grant_service_logon_rights.ps1
```

```
PS C:\Scripts\_Projet AlwaysOn> .\3.2.1.grant_service_logon_rights.ps1

Le mappage entre les noms de compte et les ID de sécurité n'a pas été effectué.

La tâche s'est terminée avec une erreur.
Lisez le fichier journal %windir%\security\logs\scesrv.log pour obtenir des informations détaillées.
Droits appliques : Log on as a service + Perform volume maintenance tasks
PS C:\Scripts\_Projet AlwaysOn> HOSTNAME
SQLS01
```

```
PS C:\Scripts\_Projet AlwaysOn> .\3.2.1.grant_service_logon_rights.ps1

Le mappage entre les noms de compte et les ID de sécurité n'a pas été effectué.

La tâche s'est terminée avec une erreur.
Lisez le fichier journal %windir%\security\logs\scesrv.log pour obtenir des informations détaillées.
Droits appliques : Log on as a service + Perform volume maintenance tasks
PS C:\Scripts\_Projet AlwaysOn> HOSTNAME
SQLS02
```

Le message « `Le mappage entre les noms de compte et les ID de sécurité n'a pas été effectué` » est un avertissement bénin émis par `secedit /configure` quand il rencontre un SID qu'il ne peut pas résoudre. Dans ce cas, c'était probablement `S-1-5-80-0` (compte système `NT SERVICE\ALL SERVICES`), qui n'est pas résolvable par `secedit` mais qui est légitime.

Le script a affiché un avertissement `secedit` bénin, mais les droits ont bien été appliqués. La vérification finale le confirme

Ce script attribue :

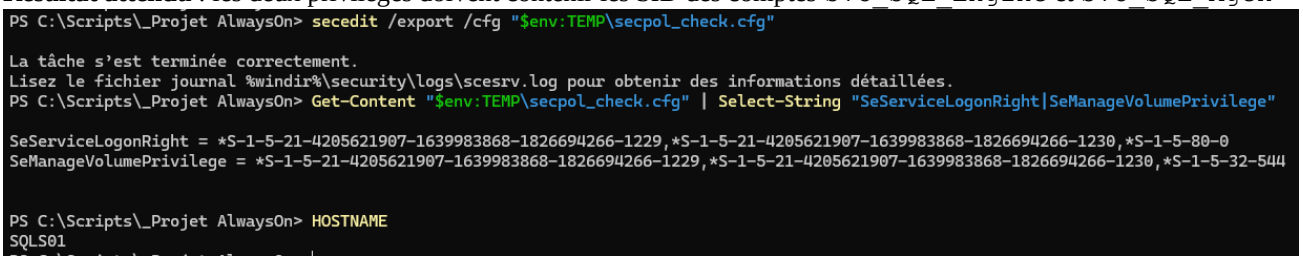
- SeServiceLogonRight (Log on as a service).
- SeManageVolumePrivilege (Perform volume maintenance tasks).

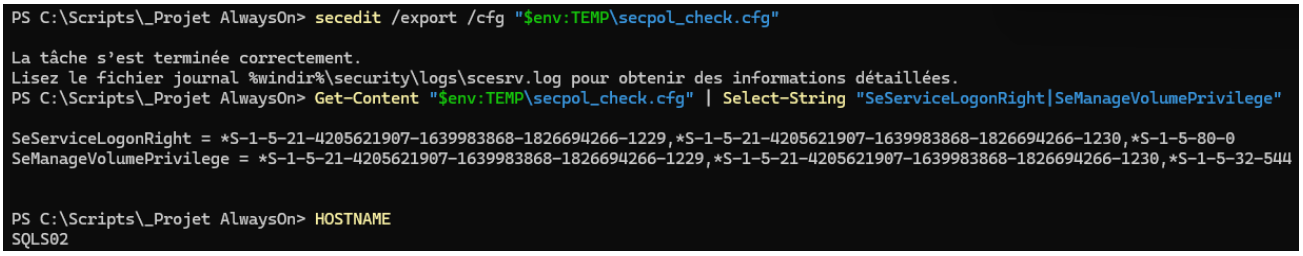
Vérification sur les deux nœuds

```
secedit /export /cfg "$env:TEMP\secpol_check.cfg"

Get-Content "$env:TEMP\secpol_check.cfg" | Select-String "SeServiceLogonRight|SeManageVolumePrivilege"
```

Résultat attendu : les deux privilèges doivent contenir les SID des comptes `SVC_SQL_Engine` et `SVC_SQL_Agen`





Sur SQLS01 et SQLS02, les privilèges exportés par **secedit** montrent :
SeServiceLogonRight = *...-1229,*...-1230,*S-1-5-80-0
SeManageVolumePrivilege = *...-1229,*...-1230,*S-1-5-32-544

Décodage :

- ...-1229 = SVC_SQL_Engine
- ...-1230 = SVC_SQL_Agent
- *S-1-5-80-0 = NT SERVICE\ALL SERVICES (compte système)
- *S-1-5-32-544 = BUILTIN\Administrateurs

Vérification sur le DC03

```
PS C:\Scripts\_Projet AlwaysOn> Get-ADUser SVC_SQL_Engine -Properties SID | Select-Object Name, SID
Name           SID
----           ---
SVC_SQL_Engine S-1-5-21-4205621907-1639983868-1826694266-1229


PS C:\Scripts\_Projet AlwaysOn> Get-ADUser SVC_SQL_Agent -Properties SID | Select-Object Name, SID
Name          SID
----          ---
SVC_SQL_Agent S-1-5-21-4205621907-1639983868-1826694266-1230
```

Explication du mécanisme des comptes de service SQL Server

Conclusion préalable

Nous venons de mettre en place le socle de sécurité sur lequel SQL Server va s'appuyer pendant toute sa durée de vie. Les deux comptes `SVC_SQL_Engine` et `SVC_SQL_Agent` ne sont pas de simples comptes AD : ce sont les **identités sous lesquelles les services SQL Server vont tourner en permanence**. Les deux privilèges qu'on vient de leur accorder (Log on as a service et Perform volume maintenance tasks) sont **indispensables** au bon fonctionnement de SQL Server, et **leur absence bloquerait l'installation ou dégraderait les performances**.

1. Les deux comptes de service

1.1 `SVC_SQL_Engine` — le moteur SQL Server

Rôle : c'est le compte sous lequel tourne le **service principal** `MSSQLSERVER`. C'est **l'identité de SQL Server lui-même**.

Ce qu'il fait :

- Démarre le service `MSSQLSERVER` au boot de la machine.
- Se connecte aux fichiers de données (`S:\SQL\Data`), de logs (`S:\SQL\Log`), de sauvegardes (`S:\SQL\Backup`) et TempDB (`S:\SQL\TempDB`).
- Écrit dans les journaux d'erreurs SQL Server.
- Se connecte aux autres nœuds du cluster Always On (endpoint 5022).
- Lit et écrit dans la base `master`, `msdb`, `model`, `tempdb`.

Sans ce compte, SQL Server ne peut **pas démarrer**.

1.2 `SVC_SQL_Agent` — l'agent SQL Server

Rôle : c'est le compte sous lequel tourne le **service `SQLSERVERAGENT`**. C'est **l'identité du planificateur de tâches SQL Server**.

Ce qu'il fait :

- Exécute les **jobs SQL Server Agent** (backups, DBCC CHECKDB, index rebuild).
- Se connecte à SQL Server via **Windows Authentication**.
- Écrit dans la base `msdb` (qui contient la définition des jobs).
- Déclenche les alertes configurées (severités 19 à 25).

Sans ce compte, SQL Server démarre mais **aucun job ne s'exécute**, et les alertes ne fonctionnent pas.

1.3 Pourquoi deux comptes distincts ?

Principe de moindre privilège : chaque service a son propre compte, avec ses propres permissions. Si `SVC_SQL_Engine` est compromis, l'attaquant n'a **pas** accès aux droits supplémentaires de `SVC_SQL_Agent`, et inversement.

Isolation : si le service SQL Agent doit être redémarré (ce qui arrive), il peut l'être sans impacter le moteur SQL Server.
Traçabilité : les événements Windows et les logs SQL Server distinguent clairement les actions du moteur et celles de l'agent.

Recommandation Microsoft : depuis SQL Server 2012, Microsoft recommande d'utiliser des **comptes de service distincts** pour le moteur et l'agent.

Voir le reste d'explication dans : « 01.Explication mécanisme SVC SQL Server.pdf » il se trouve dans « SQL-Server\AlwaysOn\Documents\Explications »

3.3 Installation de SQL Server 2022

1. Monter l'ISO (lecteur D:).
2. Choisir une installation autonome, édition Developer.
3. Instance : MSSQLSERVER (par défaut) ou OPTIMEDITSQL (nommée).
4. Collation : **SQL_Latin1_General_CP1_CI_AS**.
5. Répertoires pointés vers S:\SQL\Data / Log / Backup / TempDB.
6. Comptes de service : SVC_SQL_Engine (moteur), SVC_SQL_Agent (agent) — démarrage automatique.

Script fourni : `3.3.0.install_sql_server_2022-v2.ps1` — *exécuté sur : SQLS01 + SQLS02*

```
..
$key = Get-Content "$SecureFolder\aes.key" -Encoding Byte

$securePwd = Get-Content "$SecureFolder\sql-service-pwd.enc" | ConvertTo-SecureString -
Key $key

# Conversion en clair UNIQUEMENT pour l'argument de setup.exe

$bstr = [System.Runtime.InteropServices.Marshal]::SecureStringToBSTR($securePwd)

$plainPwd = [System.Runtime.InteropServices.Marshal]::PtrToStringAuto($bstr)

$Args = @("/Q", "/ACTION=Install", ..., "/SQLSVCPASSWORD=$plainPwd",
"/AGTSVCPASSWORD=$plainPwd", ...)

Start-Process -FilePath "$IsoDrive\Setup.exe" -ArgumentList $Args -Wait -NoNewWindow

# Nettoyage mémoire

[System.Runtime.InteropServices.Marshal]::ZeroFreeBSTR($bstr)

Remove-Variable plainPwd, securePwd, key
..
```

Remarque : Le mot de passe des comptes de service est lu depuis sql-service-pwd.enc (section 3.0). Aucun mot de passe en clair n'apparaît dans le script. Vérifier que le secret **sql-service-pwd** a bien été chiffré sur chaque nœud avant d'exécuter l'installation.

3.3.1 Vérification du prérequis :

Avant d'exécuter le script 3.3.0, vérifie sur SQLS01 et SQLS02 :

```
Test-Path "C:\ProgramData\SQLSecrets\aes.key"
Test-Path "C:\ProgramData\SQLSecrets\sql-service-pwd.enc"

Il doit retourner la valeur "True" pour chacun, Si l'un des deux est "False",
execute:
.\3.0.0.manage_secrets.ps1 -GenerateKey
.\3.0.0.manage_secrets.ps1 -SecretName "sql-service-pwd"
```

3.3.2 Points de vigilance avant exécution

Point 1 — Vérifier que l'ISO est monté sur **D: Test-Path "D:\Setup.exe"**

Point 2 — Vérifier que le module **SqlServer** est installable, Si le module n'est pas présent, le script l'installera. Mais il faut un accès Internet : **Test-NetConnection www.powershellgallery.com -Port 443**

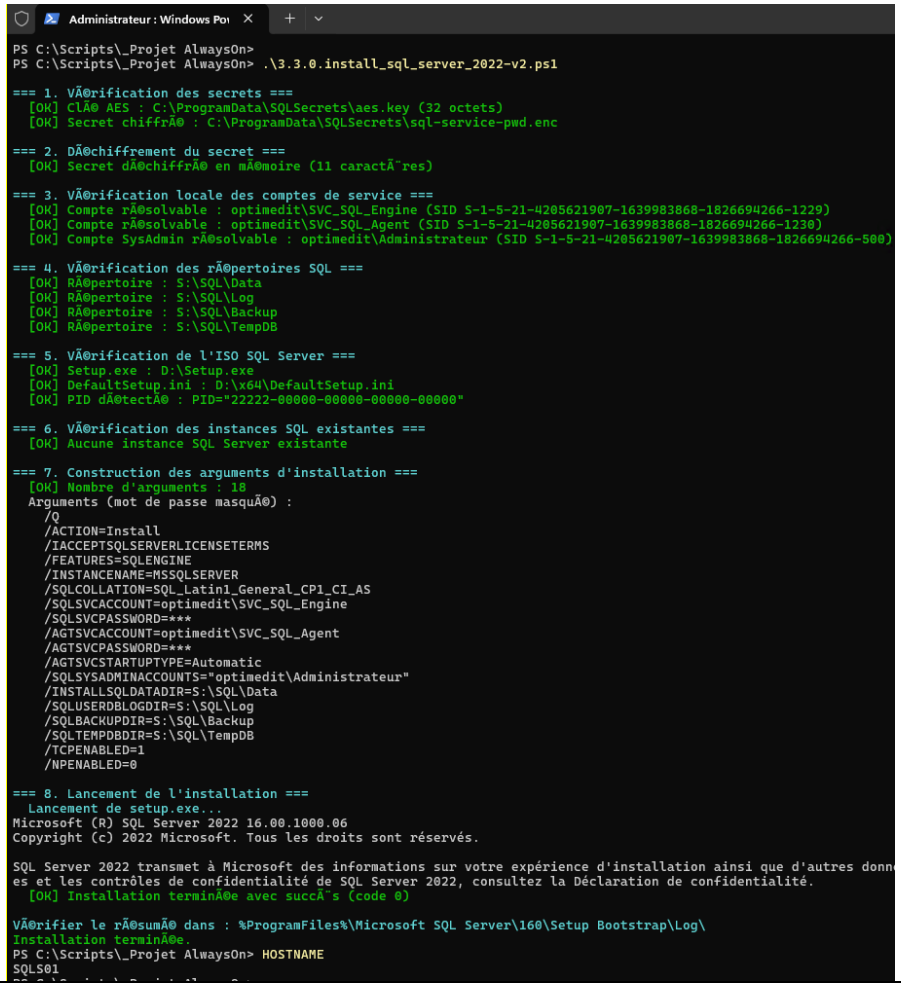
Point 3 — Durée d'installation : **L'installation prend** 10-20 minutes par nœud**. C'est normal.**

Point 4 — Logs d'installation : Les logs d'installation sont dans, **C:\Program Files\Microsoft SQL Server\160\Setup Bootstrap\Log**

EXECUTION :

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
```

```
.\3.3.0.install_sql_server_2022-v2.ps1 -WhatIf
.\3.3.0.install_sql_server_2022-v2.ps1
```



Résumé global d'installation (SQLS01 et SQLS02)

Indicateur	SQLS01	SQLS02	Statut
Résultat global	✅ Réussite	✅ Réussite	Conforme
Code de sortie	0	0	✅ Succès
Édition	Developer	Developer	✅ Conforme
Version	16 (RTM)	16 (RTM)	✅ Conforme
Durée	~6 minutes	~6 minutes	✅ Optimal
Date	24/09/2026 20:00:59	24/09/2026 20:02:38	-

Paramètres appliqués (communs aux deux nœuds)

Paramètre	Valeur	Verdict
ACTION	Install	✅

Paramètre	Valeur	Verdict
INSTANCENAME	**MSSQLSERVER** (Instance par défaut)	✅
COLLATION	SQL_Latin1_General_CP1_CI_AS	✅
SQLSVCACCOUNT	optimedit\SVC_SQL_Engine	✅
AGTSVCACCOUNT	optimedit\SVC_SQL_Agent	✅
SQLSYSADMINACCOUNTS	optimedit\Administrateur	✅
INSTALLSQLDATADIR	S:\SQL\Data	✅
SQLUSERDBLOGDIR	S:\SQL\Log	✅
SQLBACKUPDIR	S:\SQL\Backup	✅
SQLTEMPDBDIR	S:\SQL\TempDB	✅
TCPENABLED	1	✅
NPENABLED	0	✅
SQLTEMPDBFILECOUNT	8	✅ Recommandé (1 fichier par cœur)
SQLMAXDOP	8	✅ Par défaut
SQLSVCSTARTUPTYPE	Automatic	✅
AGTSVCSTARTUPTYPE	Automatic	✅
UpdateEnabled	true	✅

Avertissements :

Un seul avertissement, identique sur les deux nœuds (Avertissement IsFirewallEnabled), c'est normal. Le pare-feu Windows est actif, et les ports SQL Server (1433, 5022, 135, 445) ne sont pas encore ouverts. C'est l'objet de l'étape suivante (3.4.0.configure_network_firewall.ps1)

3.3.3 Configuration du certificat SQL server

Cette section décrit la configuration **manuelle** du certificat SSL pour SQL Server. La configuration manuelle est utilisée car le module PowerShell `SqlServer` (version 22.4.5.1) présente des limitations qui empêchent la configuration entièrement automatisée.

Prérequis :

- SQL Server 2022 installé sur le nœud.

- Compte de service `SVC_SQL_Engine` existant.
- Accès administrateur local au nœud.
- Certificat SSL présent dans `LocalMachine\My` (émis par l'AC `Optimedit-CA3`).

Procédure en 4 étapes :

1. **Ajouter Administrateur aux sysadmins SQL Server** — section 3.3.3.1
2. **Création d'un modèle de certificat** (si inexistant) — section 3.3.3.2
3. **Attribution des permissions sur la clé privée** — section 3.3.3.3
4. **Sélection du certificat dans SQL Server Configuration Manager** — section 3.3.3.4
5. **Vérification de la configuration** — section 3.3.3.5

3.3.3.1 Ajouter Administrateur aux sysadmins SQL Server

Pourquoi cette étape ?

Lors de l'installation de SQL Server, seuls les comptes spécifiés dans `/SQLSYSADMINACCOUNTS` sont ajoutés au rôle `sysadmin`. Dans notre cas, **`optimedit\Administrateur`** a été ajouté, mais **pas** le compte **local** `SQLS01\Administrateur` (ou `SQLS02\Administrateur`).

Conséquence : si un administrateur se connecte **localement** au serveur (sans être authentifié sur le domaine) et tente d'administrer SQL Server, il n'aura **pas** les droits `sysadmin`.

Solutions :

Solution 1 : on peut utiliser la commande Invoke-Sqlcmd avec la chaîne de connexion :

powershell
```
$ConnectionString = "Server=localhost;Database=master;Integrated
Security=True;Encrypt=True;TrustServerCertificate=True"

$Query = "IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name =
'SQLS01\Administrateur') BEGIN CREATE LOGIN [SQLS01\Administrateur] FROM WINDOWS;
ALTER SERVER ROLE sysadmin ADD MEMBER [SQLS01\Administrateur]; END"

Invoke-Sqlcmd -ConnectionString $ConnectionString -Query $Query
```

Solution 2 : Ou on ajoutera le compte administrateur **local** au rôle `sysadmin`.

Prérequis :

- SQL Server installé et démarré.
- Accès avec un compte déjà `sysadmin` (par exemple `optimedit\Administrateur`).

Procédure rajout le compte **administrateur local** au rôle **sysadmin** sur les deux nœuds, Sur SQLS01/ SQLS02, en tant que optimedit\Administrateur (ou un compte sysadmin), on exécute :

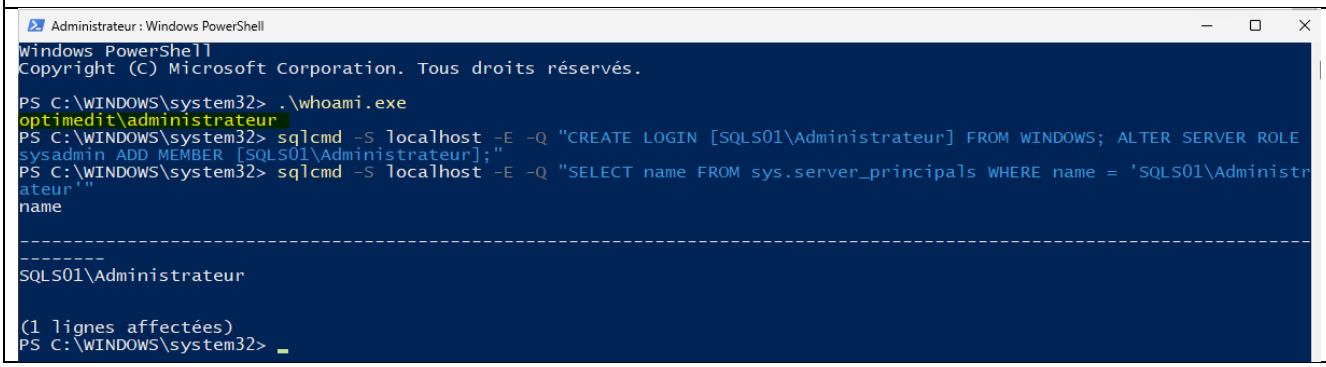
Sur SQLS01
```
sqlcmd -S localhost -E -Q "CREATE LOGIN [SQLS01\Administrateur] FROM WINDOWS;
ALTER SERVER ROLE sysadmin ADD MEMBER [SQLS01\Administrateur];"
```
Vérification
```
sqlcmd -S localhost -E -Q "SELECT name FROM sys.server_principals WHERE name =
'SQLS01\Administrateur'"
```

Sur SQLS02

```
sqlcmd -S localhost -E -Q "CREATE LOGIN [SQLS02\Administrateur] FROM WINDOWS;
ALTER SERVER ROLE sysadmin ADD MEMBER [SQLS02\Administrateur];"
```

Vérification

```
sqlcmd -S localhost -E -Q "SELECT name FROM sys.server_principals WHERE name =
'SQLS02\Administrateur'"
```



3.3.3.2 Création d'un modèle de certificat auto-enrôlement

Cette section décrit la création d'un modèle de certificat auto-enrôlement pour les serveurs SQL. **Si un modèle équivalent existe déjà** (par exemple `Ansible-WinRM-FQDN-SERVERS`), **passer directement à la section 3.3.3.3**.

Prérequis :

- Le rôle **AD CS** (Active Directory Certificate Services) doit être installé sur `DC03`.
- Accès administrateur à l'Autorité de certification.

Procédure sur DC03 :

1. Ouvre **`certsrv.msc`** (Autorité de certification).
2. **Clic droit** sur **Modèles de certificats → Gérer**.
3. **Clic droit** sur un modèle existant (par exemple `Ordinateur`) → **Dupliquer le modèle**.
4. Configure les onglets suivants :

   Onglet Général :

   - Nom complet : `SQLServer-SSL-FQDN-SERVERS`
   - Nom du modèle : `SQLServer-SSL-FQDN-SERVERS`
   - Période de validité : **2 années**
   - Période de renouvellement : **6 semaines**
   - Publier le certificat dans Active Directory : **coché**
   - Ne pas utiliser la réinscription automatique si un certificat dupliqué existe dans AD : **non coché**

   Onglet Nom du sujet :

   - Construire à partir de ces informations Active Directory : **sélectionné**
   - Format du nom du sujet : **`Nom commun`**

- Inclure cette information dans le nom de substitution du sujet : **Nom DNS** **coché**

Onglet Traitement de la demande :

- Objet : **`Signature et chiffrement`**
- Autoriser l'exportation de la clé privée : **non coché**
- Inscrire le sujet sans exiger une entrée de la part de l'utilisateur : **sélectionné** (grisée)

Onglet Chiffrement :

- Catégorie de fournisseur : `Fournisseur de stockage de clés`
- Nom de l'algorithme : `RSA`
- Taille de clé minimale : `2048`
- Hachage de la demande : `SHA256`
- Dans « choisissez les fournisseurs de chiffrement pouvant être utilisés » cocher : **« Microsoft Software Key Storage Provider » (KSP ici = CNG** pas un **CSP** classique)
- Hachage de la demande : SHA512

Onglet Compatibilité :

- Autorité de certification : `Windows Server 2016`
- Destinataire du certificat : `Windows 10/Windows Server 2016`

Onglet Extensions → Stratégies d'application :

- `Authentification du serveur`
- `Authentification du client`

Onglet Sécurité :

- Ajouter `Ordinateurs du domaine` avec les droits : **`Lecture`**, **`Inscription`**, **`Inscription automatique`**
- Vérifier que `Admins du domaine` a **`Contrôle total`**

5. **Appliquer → OK**.
6. **Publier le modèle** : clic droit sur **Modèles de certificats → Nouveau → Modèle de certificat à délivrer →** sélectionner `SQLServer-SSL-FQDN-SERVERS`.
7. **Redémarrer le service AD CS** : `Restart-Service certsvc`.

Vérification :

Sur DC03, exécute :

powershell
```
certutil -CATemplates | Select-String "SQLServer-SSL"
```

Résultat attendu : le modèle `SQLServer-SSL-FQDN-SERVERS` doit apparaître.

Note importante : *Le modèle `Ansible-WinRM-FQDN-SERVERS` existant dans l'infrastructure `optimedit.eu` est parfaitement utilisable pour SQL Server. Il contient les mêmes extensions (Authentification du serveur + client) et utilise le même KSP.*

3.3.3.3 Attribution des permissions sur la clé privée

Sur SQLS01 et SQLS02 :

Script fourni : 3.3.3.3.CheckPrivateKeyPermissions.ps1

1. Ouvre **`certlm.msc`** (Gestionnaire de certificats - Ordinateur local).
2. **Personnel → Certificats**.
3. Trouve le certificat `CN=<NOM_DU_SERVEUR>.optimedit.eu` (par exemple `CN=SQLS01.optimedit.eu`).
4. **Clic droit → Toutes les tâches → Gérer les clés privées**.
5. Clique sur **Ajouter**.
6. Saisis **`SVC_SQL_Engine`** (ou `optimedit\SVC_SQL_Engine`).
7. Clique sur **OK**.
8. Sélectionne **`SVC_SQL_Engine`** dans la liste.
9. Coche **Contrôle total** (ou au minimum **Lecture**).
10. **Appliquer → OK**.

Vérification : Sur SQLS01, exécute :

```
# 3.3.3.3.CheckPrivateKeyPermissions.ps1
$Hostname = $env:COMPUTERNAME
$Cert = Get-ChildItem Cert:\LocalMachine\My | Where-Object { $_.Subject -like
"*$Hostname.optimedit.eu*" }

Write-Host "Certificat : $($Cert.Subject)"
Write-Host "Thumbprint : $($Cert.Thumbprint)"
Write-Host ""

$rsa =
[System.Security.Cryptography.X509Certificates.RSACertificateExtensions]::GetRSAPrivateKey
($Cert)

if ($rsa -is [System.Security.Cryptography.RSACng]) {
    Write-Host "Type : CNG (Key Storage Provider)" -ForegroundColor Cyan
    $uniqueName = $rsa.Key.UniqueName
} else {
    Write-Host "Type : CSP (Cryptographic Service Provider)" -ForegroundColor Cyan
    $uniqueName = $rsa.CspKeyContainerInfo.UniqueKeyContainerName
}

Write-Host "Nom de clé : $uniqueName"
Write-Host ""

$keyFile = Join-Path "C:\ProgramData\Microsoft\Crypto\Keys" $uniqueName
Write-Host "Fichier de clé : $keyFile"
Write-Host "Existe : $(Test-Path $keyFile)"
Write-Host ""

if (Test-Path $keyFile) {
    Write-Host "Permissions :" -ForegroundColor Yellow
    Get-Acl $keyFile | Select-Object -ExpandProperty Access |
        Format-Table IdentityReference, FileSystemRights, AccessControlType -AutoSize
}
```

Résultat attendu : une ligne avec `SVC_SQL_Engine` et `FullControl` ou `Read`.

3.3.3.4 Sélection du certificat dans SQL Server Configuration Manager

Sur SQLS01 et SQLS02 :

1. Ouvre **SQL Server Configuration Manager SSCM**.

2. **Configuration réseau SQL Server → Protocoles pour MSSQLSERVER → Clic droit → Propriétés**.
3. Onglet **Certificat**.
4. Dans le menu déroulant **Certificat**, sélectionne **`<NOM_DU_SERVEUR>.optimedit.eu`** (par exemple `SQLS01.optimedit.eu`).
5. **Appliquer → OK**.
6. **Redémarrer SQL Server** :

   powershell

   ```
   Restart-Service MSSQLSERVER -Force
   ```

Note : *Le redémarrage est nécessaire pour que SQL Server recharge la configuration du certificat.*

3.3.3.5 Vérification de la configuration

Script fourni de vérification 3.3.3.5.CheckSqlCertificate.ps1
— exécuté sur : SQLS01 + SQLS02

Le script 3.3.3.5.CheckSqlCertificate.ps1 automatise les 4 vérifications ci-dessous. Il est idempotent et sans risque : il ne modifie rien, il lit uniquement.

PowerShell
```
.\3.3.3.5.CheckSqlCertificate.ps1
```

Ou exécuter manuellement les commandes suivantes **Sur SQLS01 et SQLS02** pour vérifier chaque aspect de la configuration.

Vérification 1 — Certificat dans `LocalMachine\My` :
powershell
```
$Cert = Get-ChildItem Cert:\LocalMachine\My | Where-Object { $_.Subject -like
"*$env:COMPUTERNAME.optimedit.eu*" }


$Cert | Format-List Subject, Issuer, Thumbprint, HasPrivateKey, NotAfter
```

Vérification 2 — Valeur `Certificate` dans le registre :
powershell
```
$RegPath = "HKLM:\SOFTWARE\Microsoft\Microsoft SQL
Server\MSSQL16.MSSQLSERVER\MSSQLServer\SuperSocketNetLib"

Get-ItemProperty -Path $RegPath -Name "Certificate" | Select-Object Certificate
```

Résultat attendu : le thumbprint du certificat (32 caractères hexadécimaux).

Vérification 3 — Certificat présenté par SQL Server :
powershell
```
$TcpClient = New-Object System.Net.Sockets.TcpClient

$TcpClient.Connect("localhost", 1433)
$SslStream = New-Object System.Net.Security.SslStream($TcpClient.GetStream(),
$false, ({$true}))
$SslStream.AuthenticateAsClient("$env:COMPUTERNAME.optimedit.eu")
```

```
$PresentedCert = $SslStream.RemoteCertificate
Write-Host "Sujet      : $($PresentedCert.Subject)"
Write-Host "Émetteur   : $($PresentedCert.Issuer)"
Write-Host "Thumbprint: $($PresentedCert.GetCertHashString())"
$SslStream.Close()
$TcpClient.Close()
```

Vérification 4 — Connexion chiffrée :

powershell

```
$ConnectionString = "Server=localhost;Database=master;Integrated
Security=True;Encrypt=True;TrustServerCertificate=True"

Invoke-Sqlcmd -ConnectionString $ConnectionString -Query "SELECT
SERVERPROPERTY('Edition') AS Edition, session_id, encrypt_option FROM
sys.dm_exec_connections WHERE session_id = @@SPID"
```

Résultat attendu : `Developer Edition (64-bit)` et `encrypt_option = TRUE`.

3.3.3.6 Application et redémarrage du certificat

Script fourni **3.3.3.6.RestartSqlCertificate.ps1**
— exécuté sur : SQLS01 + SQLS02

Le script 3.3.3.6.Restart-SqlCertificate.ps1 applique le certificat SSL configuré dans le registre et force SQL Server à le recharger, puis vérifie que le certificat est bien présenté.

PowerShell

```
.\ 3.3.3.6.RestartSqlCertificate.ps1
```

3.3.4 Procédure de vérification après l'installation

3.3.4.1 Vérifier que les services sont démarrés sur SQLS01 et SQLS02

```
Get-Service MSSQLSERVER | Select-Object Name, Status, StartType
Get-Service SQLSERVERAGENT | Select-Object Name, Status, StartType
```

3.3.4.2 Vérifier la version de SQL Server

```
Invoke-Sqlcmd -Query "SELECT @@VERSION AS Version" -ServerInstance "localhost"
```

Ou

```
Invoke-Sqlcmd -Query "SELECT @@VERSION AS Version" -ServerInstance "FQDN du nœud"
```

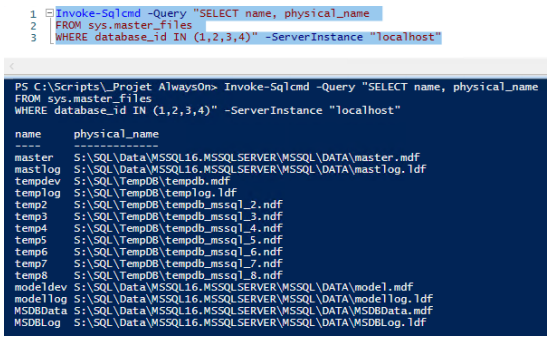
Note : Invoke-Sqlcmd nécessite le module SqlServer. S'il n'est pas installé, utilise SSMS (voir section 4).

3.3.4.3 Vérifier les emplacements des fichiers sur SQLS01 et SQLS02

```
Invoke-Sqlcmd -Query "SELECT name, physical_name FROM sys.master_files WHERE database_id IN (1,2,3,4)" -ServerInstance "localhost"
```

```
Invoke-Sqlcmd -Query "SELECT name, physical_name
FROM sys.master_files
WHERE database_id IN (1,2,3,4)" -ServerInstance "localhost"
```

Remarque : si erreur avec cible "localhost" préciser le nom du serveur « SQLS02.optimedit.eu.



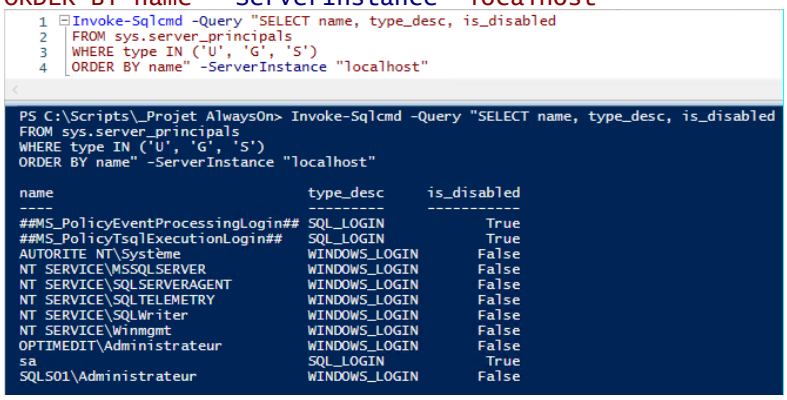
```
PS C:\Scripts\_Projet AlwaysOn> Invoke-Sqlcmd -Query "SELECT name, physical_name
FROM sys.master_files
WHERE database_id IN (1,2,3,4)" -ServerInstance "localhost"

name     physical_name
----     -------------
master   S:\SQL\Data\MSSQL16.MSSQLSERVER\MSSQL\DATA\master.mdf
mastlog  S:\SQL\Data\MSSQL16.MSSQLSERVER\MSSQL\DATA\mastlog.ldf
tempdev  S:\SQL\TempDB\tempdb.mdf
templog  S:\SQL\TempDB\templog.ldf
temp2    S:\SQL\TempDB\tempdb_mssql_2.ndf
temp3    S:\SQL\TempDB\tempdb_mssql_3.ndf
temp4    S:\SQL\TempDB\tempdb_mssql_4.ndf
temp5    S:\SQL\TempDB\tempdb_mssql_5.ndf
temp6    S:\SQL\TempDB\tempdb_mssql_6.ndf
temp7    S:\SQL\TempDB\tempdb_mssql_7.ndf
temp8    S:\SQL\TempDB\tempdb_mssql_8.ndf
modeldev S:\SQL\Data\MSSQL16.MSSQLSERVER\MSSQL\DATA\model.mdf
modellog S:\SQL\Data\MSSQL16.MSSQLSERVER\MSSQL\DATA\modellog.ldf
MSDBData S:\SQL\Data\MSSQL16.MSSQLSERVER\MSSQL\DATA\MSDBData.mdf
MSDBLog  S:\SQL\Data\MSSQL16.MSSQLSERVER\MSSQL\DATA\MSDBLog.ldf
```

3.3.4.4 Vérifier les logins SQL Server SQLS01 et SQLS02

```
Invoke-Sqlcmd -Query "SELECT name, type_desc, is_disabled FROM sys.server_principals WHERE type IN ('U', 'G', 'S') ORDER BY name" -ServerInstance "localhost"
```

```
Invoke-Sqlcmd -Query "SELECT name, type_desc, is_disabled
FROM sys.server_principals
WHERE type IN ('U', 'G', 'S')
ORDER BY name" -ServerInstance "localhost"
```



```
PS C:\Scripts\_Projet AlwaysOn> Invoke-Sqlcmd -Query "SELECT name, type_desc, is_disabled
FROM sys.server_principals
WHERE type IN ('U', 'G', 'S')
ORDER BY name" -ServerInstance "localhost"

name                                type_desc     is_disabled
----                                ---------     -----------
##MS_PolicyEventProcessingLogin##   SQL_LOGIN            True
##MS_PolicyTsqlExecutionLogin##     SQL_LOGIN            True
AUTORITE NT\Système                 WINDOWS_LOGIN       False
NT SERVICE\MSSQLSERVER              WINDOWS_LOGIN       False
NT SERVICE\SQLSERVERAGENT           WINDOWS_LOGIN       False
NT SERVICE\SQLTELEMETRY             WINDOWS_LOGIN       False
NT SERVICE\SQLWriter                WINDOWS_LOGIN       False
NT SERVICE\Winmgmt                  WINDOWS_LOGIN       False
OPTIMEDIT\Administrateur            WINDOWS_LOGIN       False
sa                                  SQL_LOGIN            True
SQLS01\Administrateur               WINDOWS_LOGIN       False
```

3.3.4.5 Vérifier les rôles serveur SQLS01 et SQLS02

Une seule ligne :

```
Invoke-Sqlcmd -Query "SELECT r.name AS RoleName, m.name AS MemberName FROM sys.server_role_members rm JOIN sys.server_principals r ON rm.role_principal_id = r.principal_id JOIN sys.server_principals m ON rm.member_principal_id = m.principal_id WHERE r.name IN ('sysadmin', 'securityadmin', 'serveradmin') ORDER BY r.name, m.name" -ServerInstance "localhost"
```

```
Invoke-Sqlcmd -Query "SELECT r.name AS RoleName, m.name AS MemberName
    FROM sys.server_role_members rm
    JOIN sys.server_principals r ON rm.role_principal_id = r.principal_id
    JOIN sys.server_principals m ON rm.member_principal_id = m.principal_id
    WHERE r.name IN ('sysadmin', 'securityadmin', 'serveradmin')
    ORDER BY r.name, m.name" -ServerInstance "localhost"
```

```
Invoke-Sqlcmd -Query "SELECT r.name AS RoleName, m.name AS MemberName
    FROM sys.server_role_members rm
    JOIN sys.server_principals r ON rm.role_principal_id = r.principal_id
    JOIN sys.server_principals m ON rm.member_principal_id = m.principal_id
    WHERE r.name IN ('sysadmin', 'securityadmin', 'serveradmin')
    ORDER BY r.name, m.name" -ServerInstance "localhost"
```

```
PS C:\Scripts\_Projet AlwaysOn> HOSTNAME.EXE
SQLS01

PS C:\Scripts\_Projet AlwaysOn> whoami.exe
sqls01\administrateur

PS C:\Scripts\_Projet AlwaysOn> Invoke-Sqlcmd -Query "SELECT r.name AS RoleName, m
    FROM sys.server_role_members rm
    JOIN sys.server_principals r ON rm.role_principal_id = r.principal_id JOIN sys
    WHERE r.name IN ('sysadmin', 'securityadmin', 'serveradmin')
    ORDER BY r.name, m.name" -ServerInstance "localhost"

RoleName MemberName
-------- ----------
sysadmin NT SERVICE\MSSQLSERVER
sysadmin NT SERVICE\SQLSERVERAGENT
sysadmin NT SERVICE\SQLWriter
sysadmin NT SERVICE\Winmgmt
sysadmin OPTIMEDIT\Administrateur
sysadmin sa
sysadmin SQLS01\Administrateur
```

> On peut aussi faire ces vérifications soit via SSMS ou via SQL Server Configuration Manager.
>
> **Pour ce fait, consulter le document « 02.Vérification post installation SQL Server.pdf » qui se trouve dans « « SQL-Server\AlwaysOn\Documents\Explications »**

3.4 Configuration réseau

Avant de commencer 3.4.0

```
# Les services SQL Server sont Running
Get-Service MSSQLSERVER, SQLSERVERAGENT | Select-Object Name, Status
# Le pare-feu Windows est actif
Get-NetFirewallProfile | Select-Object Name, Enabled
```

Flux	Port
SQL Server	1433/TCP
Endpoints Always On	5022/TCP
RPC cluster	135/TCP
SMB (witness)	445/TCP

Script fourni : `3.4.0.configure_network_firewall.ps1` — *exécuté sur : SQLS01 + SQLS02*

```
New-NetFirewallRule -DisplayName "SQL Server - 1433" -Direction Inbound -Protocol TCP -
LocalPort 1433 -Action Allow
New-NetFirewallRule -DisplayName "AlwaysOn Endpoint - 5022" -Direction Inbound -Protocol
TCP -LocalPort 5022 -Action Allow
New-NetFirewallRule -DisplayName "Cluster RPC - 135" -Direction Inbound -Protocol TCP -
LocalPort 135 -Action Allow
New-NetFirewallRule -DisplayName "SMB - 445" -Direction Inbound -Protocol TCP -LocalPort
445 -Action Allow
```

```
  Regle creee : SQL Server - 1433 (port 1433)
  Regle creee : AlwaysOn Endpoint - 5022 (port 5022)
  Regle creee : Cluster RPC - 135 (port 135)
  Regle creee : SMB - 445 (port 445)

Configuration reseau terminee.
VA©rifier avec la commande suivante la configuration:
Get-NetFirewallRule -DisplayName "SQL Server - 1433","AlwaysOn Endpoint - 5022","Cluster RPC - 135","SMB - 445"

DisplayName              Enabled Direction Action
-----------              ------- --------- ------
SQL Server - 1433           True   Inbound  Allow
AlwaysOn Endpoint - 5022    True   Inbound  Allow
Cluster RPC - 135           True   Inbound  Allow
SMB - 445                   True   Inbound  Allow


PS C:\Users\Administrateur> HOSTNAME.EXE
SQLSO1
```

```
  Regle creee : SQL Server - 1433 (port 1433)
  Regle creee : AlwaysOn Endpoint - 5022 (port 5022)
  Regle creee : Cluster RPC - 135 (port 135)
  Regle creee : SMB - 445 (port 445)

Configuration reseau terminee.
VA©rifier avec la commande suivante la configuration:
Get-NetFirewallRule -DisplayName "SQL Server - 1433","AlwaysOn Endpoint - 5022","Cluster RPC - 135","SMB - 445"

DisplayName              Enabled Direction Action
-----------              ------- --------- ------
SQL Server - 1433           True   Inbound  Allow
AlwaysOn Endpoint - 5022    True   Inbound  Allow
Cluster RPC - 135           True   Inbound  Allow
SMB - 445                   True   Inbound  Allow


PS C:\Scripts\_Projet AlwaysOn> HOSTNAME.EXE
SQLSO2
```

3.5 Récapitulatifs (pré-vol & phase 1)

1.1 Infrastructure et prérequis

#	Étape	Statut
1	Serveur HYPER-V1 vérifié (RAM 144 GB, 40 threads, CPU Xeon Silver 4210R)	✅
2	VMs SQLS01 et SQLS02 créées et configurées (10 GB RAM, 4 cœurs, disque S: 65 GB)	✅
3	DC03 opérationnel (contrôleur de domaine optimedit.eu)	✅
4	OPT-FS03 opérationnel (serveur de fichiers)	✅
5	OPT-RP-01 opérationnel (reverse proxy ARR)	✅
6	6 serveurs IIS backends opérationnels	✅
7	DNS configuré (13 sites pointent vers OPT-RP-01)	✅
8	WinRM configuré sur tous les serveurs	✅

1.2 Phase 1 — Préparation des serveurs SQL

#	Étape	Script	Statut	Cible
1	Gestion des secrets (clé AES 256 bits + sql-service-pwd.enc)	3.0.0.manage_secrets.ps1	✅	SQLS01 + SQLS02
2	Création des dossiers SQL (S:\SQL\Data, Log, Backup, TempDB, Backup\Audit)	3.1.0.create_storage_directories.ps1	✅	SQLS01 + SQLS02
3	Création des comptes de service AD (SVC_SQL_Engine, SVC_SQL_Agent)	3.2.0.create_sql_service_accounts.ps1	✅	DC03
4	Attribution des droits locaux (SeServiceLogonRight, SeManageVolumePrivilege)	3.2.1.grant_service_logon_rights.ps1	✅	SQLS01 + SQLS02
5	Installation SQL Server 2022 Developer	3.3.0.install_sql_server_2022-v2.ps1	✅	SQLS01 + SQLS02
6	Installation du module PowerShell SqlServer (version 22.4.5.1)	3.3.1.install_sqlserver_module.ps1	✅	SQLS01 + SQLS02

#	Étape	Script	Statut	Cible
7	Ajout des admins locaux aux sysadmins SQL (SQLS01\Administrateur, SQLS02\Administrateur)	-	✅	SQLS01 + SQLS02
8	Attribution des permissions sur la clé privée du certificat (SVC_SQL_Engine = FullControl)	3.3.3.3.CheckPrivateKeyPermissions.ps1	✅	SQLS01 + SQLS02
9	Configuration du certificat SSL (CN=SQLS01.optimedit.eu / CN=SQLS02.optimedit.eu)	3.3.3.4 (manuel via SSCM)	✅	SQLS01 + SQLS02
10	Vérification du certificat SSL (My, registre, SSL direct, connexion chiffrée)	3.3.3.5.CheckSqlCertificate.ps1	✅	SQLS01 + SQLS02
11	Application et redémarrage du certificat SSL	3.3.3.6.RestartSqlCertificate.ps1	✅	SQLS01 + SQLS02
12	Configuration réseau / pare-feu (ports 1433, 5022, 135, 445)	3.4.0.configure_network_firewall.ps1	✅	SQLS01 + SQLS02

1.3 État final des nœuds SQL

Élément	SQLS01	SQLS02
SQL Server 2022 Developer	✅ Installé	✅ Installé
Service MSSQLSERVER	✅ Running	✅ Running
Service SQLSERVERAGENT	✅ Running	✅ Running
Certificat SSL (CN=SQLS01/02.optimedit.eu)	✅ Configuré	✅ Configuré
Permissions clé privée (SVC_SQL_Engine)	✅ FullControl	✅ FullControl
Connexion chiffrée (encrypt_option = TRUE)	✅ Validée	✅ Validée
Registre Certificate	✅ 50ab565a...	✅ 0a8d26fb...
Pare-feu (ports 1433, 5022, 135, 445)	✅ Ouvert	✅ Ouvert

4. Phase 2 — Cluster Windows

Prérequis à vérifier avant de commencer

Services SQL Server

```
Get-Service MSSQLSERVER, SQLSERVERAGENT | Select-Object Name, Status
```

Appartenance au domaine

```
(Get-CimInstance Win32_ComputerSystem).Domain
```

Version de Windows

```
(Get-CimInstance Win32_OperatingSystem).Caption
```

Ressources (RAM, CPU)

```
Get-CimInstance Win32_ComputerSystem | Select-Object TotalPhysicalMemory
Get-CimInstance Win32_Processor | Select-Object NumberOfCores
```

Vérifications sur le réseau, depuis SQLS01, vérifie que SQLS02 est joignable

```
Test-NetConnection SQLS02 -Port 1433
Test-NetConnection SQLS02 -Port 5022
```

Le port 5022 n'est pas joignable parce que SQL Server n'écoute pas encore sur ce port, il suffit de faire une vérification d'ouverture de port sur SQLS02, en plus ce port n'est pas nécessaire pour la Phase 2. Il sera utilisé en Phase 3, après la création de l'endpoint Always On:

```
Get-NetFirewallRule -DisplayName "AlwaysOn Endpoint - 5022" | Select-Object
DisplayName, Enabled, Direction, Action
```

Vérifications des ports du cluster, sur SQLS01 et SQLS02, vérifie que les ports 135 et 445 sont ouverts

```
Get-NetFirewallRule -DisplayName "Cluster RPC - 135","SMB - 445" | Select-Object
DisplayName, Enabled, Direction, Action
```

Vérification de l'adresse IP 192.168.1.253 si libre

4.1 Installation de la fonctionnalité

Script fourni : `4.1.0.install_failover_clustering_feature.ps1` — *exécuté sur : SQLS01 + SQLS02*

```
Install-WindowsFeature Failover-Clustering -IncludeManagementTools
Install-WindowsFeature RSAT-Clustering
Install-WindowsFeature RSAT-AD-PowerShell
```

Exécutions sur SQLS01 et SQLS02
Séquence recommandée :

1. **Lance `4.1.0` sur SQLS01 et SQLS02 en parallèle.**

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
.\4.1.0.install_failover_clustering_feature.ps1
```

2. **Attends la fin des deux installations.**
3. **Redémarre SQLS01 et SQLS02** (en parallèle ou l'un après l'autre, si « **Restart Needed: No** » n'a pas besoin de redémarrer).
4. **Après redémarrage**, vérifie que les rôles sont installés

```
Get-WindowsFeature Failover-Clustering, RSAT-Clustering | Select-Object Name,
InstallState
```

Resulta :

```
Name                InstallState
----                ------------
Failover-Clustering    Installed
RSAT-Clustering        Installed
RSAT-AD-PowerShell     Installed
```

5. **Passe à `4.2.0`** (validation du cluster).

```
PS C:\WINDOWS\system32> Set-Location "C:\Scripts\_Projet AlwaysOn"
PS C:\Scripts\_Projet AlwaysOn> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
Success Restart Needed Exit Code      Feature Result
------- -------------- ---------      --------------
Success Restart Needed Exit Code      Feature Result
------- -------------- ---------      --------------
True    No             Success        {Clustering de basculement, Outils d'admin...
True    No             NoChangeNeeded {}
True    No             Success        {Module Active Directory pour Windows Powe...
Fonctionnalites de clustering installees.
                                HOSTNAME.EXE
SQLS01
```

```
PS C:\WINDOWS\system32> Set-Location "C:\Scripts\_Projet AlwaysOn"
PS C:\Scripts\_Projet AlwaysOn> Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
PS C:\Scripts\_Projet AlwaysOn> .\4.1.0.install_failover_clustering_feature.ps1

Success Restart Needed Exit Code      Feature Result
------- -------------- ---------      --------------
True    No             Success        {Clustering de basculement, Outils d'admin...
True    No             NoChangeNeeded {}
True    No             Success        {Module Active Directory pour Windows Powe...
Fonctionnalites de clustering installees.


PS C:\Scripts\_Projet AlwaysOn> HOSTNAME.EXE
SQLS02
```

4.2 Validation du cluster

Vérification des commandes de clustering sur les deux nœuds :

```
Get-Command Get-Cluster, New-Cluster, Test-Cluster | Select-Object Name, Source
```

Résultat :

```
Name         Source
----         ------
Get-Cluster  FailoverClusters
New-Cluster  FailoverClusters
Test-Cluster FailoverCluster
```

Script fourni : `4.2.0.validate_cluster.ps1` — *exécuté sur : SQLS01 ou SQLS02*

Exécuter le script ou la commande suivante :

```
Test-Cluster -Node "SQLS01","SQLS02" -Verbose
```

💡 **Conseil** : Test-Cluster génère un rapport HTML dans %SystemRoot%\Cluster\Reports\. Ouvrez-le et vérifiez qu'il n'y a pas d'erreur bloquante (les avertissements de type « **réseau non redondant** » , et « sans stockage partagé » sont normaux dans un labo).

Verdict global : ✅ Le cluster peut être créé, aucune erreur bloquant, rapport dans « SQL-Server\AlwaysOn\Rapports\4.2.0 »

Points de validation clés

Élément de validation	Statut	Détails / Configuration
Éditions OS	✅ Validé	Windows Server 2025 Datacenter Evaluation (sur les deux nœuds)
Version OS	✅ Validé	10.0.26100 (identique sur SQLS01 et SQLS02)
Domaine / OU	✅ Validé	optimedit.eu, conteneur OU=SQL Server
Architecture CPU	✅ Validé	x64 (sur les deux nœuds)
Pilotes signés	✅ Validé	Uniquement des pilotes signés
Services indispensables	✅ Validé	RPC, RemoteRegistry, LanmanServer, LanmanWorkstation, WinMgmt (tous en *Running / Automatic*)
Mises à jour logicielles	✅ Validé	Identiques sur les deux nœuds
Pare-feu Windows	✅ Validé	Configuré pour autoriser la communication du cluster
Configuration IP	✅ Validé	Pas de doublon, sous-réseaux cohérents (192.168.1.0/24)
Réseau cluster	✅ Validé	*Cluster Network 1* créé (préfixe 192.168.1.0/24, DHCP désactivé)
Image mémoire (Crash dump)	✅ Validé	Réglée sur *Automatic* sur les deux nœuds

Configuration à appliquer sur le réseau 192.168.3.0/24 dans le Gestionnaire de cluster :

Une interface réseau rajouté sur chaque nœud sur le réseau 192.168.3.0/24 :

192.168.3.251 SQLS01
192.1683.252 SQLS02

- **Rôle :** `Internal cluster communications only` (Communication interne du cluster uniquement).
- **DNS :** Ne doit **pas** être enregistré dans le DNS.
- **Passerelle par défaut :** Aucune. Ce réseau ne doit pas pouvoir router vers Internet ou d'autres sous-réseaux.

4.3 Création du cluster

- Nom du cluster : **CL-SQL-AG**
- Adresse IP du cluster (CNO) : 192.168.1.253

Script fourni : `4.3.0.create_cluster.ps1` — *exécuté sur : SQLS01 ou SQLS02*

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
.\4.1.0.install_failover_clustering_feature.ps1

Ou juste la commande:
New-Cluster -Name "CL-SQL-AG" -Node "SQLS01","SQLS02" -StaticAddress
"192.168.1.253" -NoStorage
```

Voir les réseaux du cluster

```
Get-ClusterNetwork
```

Configurer le réseau principal (192.168.1.x) pour le client et le cluster

`(Get-ClusterNetwork | Where-Object { $_.Address -eq "192.168.1.0" }).Role = 3` # 3 = Client and Cluster

Configurer le réseau secondaire (192.168.3.x) en interne uniquement

`(Get-ClusterNetwork | Where-Object { $_.Address -eq "192.168.3.0" }).Role = 1` # 1 = Internal

4.3.1 Création du partage ClusterWitness (sur OPT-FS03)

Script fourni : `4.3.1.create_cluster_witness_share.ps1` — *exécuté sur : OPT-FS03*

Avant de configurer le quorum du cluster (section 4.4), le partage \\OPT-FS03\ClusterWitness doit exister. Ce partage servira de File Share Witness (témoin de quorum) pour le cluster CL-SQL-AG.

Prérequis : *Le cluster **CL-SQL-AG** doit exister dans Active Directory (créé par 4.3.0.create_cluster.ps1). Sans cela, le compte ordinateur **CL-SQL-AG$** n'existe pas et les droits ne peuvent pas être accordés.*
Le volume P: doit exister sur OPT-FS03.
Le script doit être exécuté en administrateur local sur OPT-FS03.

```
Set-Location "C:\Scripts\_Projet AlwaysOn"
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force

# Prévisualisation
.\4.3.1.create_cluster_witness_share.ps1 -WhatIf

# Le script va chercher le SID tout seul
.\4.3.1.create_cluster_witness_share.ps1

# Ou avec un autre nom de cluster si besoin
.\4.3.1.create_cluster_witness_share.ps1 -ClusterName "autre nom que CL-SQL-AG"

# Ou aller chercher le SID sur DC03, puis le copier-coller
# Prévisualisation
.\4.3.1.create_cluster_witness_share.ps1 -ClusterSID "S-1-5-21-4205621907-1639983868-
1826694266-1231" -WhatIf

# Exécution réelle
.\4.3.1.create_cluster_witness_share.ps1 -ClusterSID "S-1-5-21-4205621907-1639983868-
1826694266-1231"
```

Explication Comment ça Marche : la Fonction Get-ClusterComputerSID du script 4.3.1.create_cluster_witness_share.ps1

Voir document : **AlwaysOn\Documents\Explications\4.3.1.FonctionGet-ClusterComputerSID.pdf**

Vérification Avant de passer à l'étape 4.4

```
Sur le serveur de fichier
Write-Host "=== Vérification finale du partage témoin ===" -ForegroundColor Cyan
Write-Host ""

# 1. Dossier
Write-Host "1. Dossier :" -ForegroundColor Yellow
$folderExists = Test-Path "P:\ClusterWitness"
Write-Host "   P:\ClusterWitness : $(if ($folderExists) { 'OK' } else { 'ABSENT' })" -
ForegroundColor $(if ($folderExists) { 'Green' } else { 'Red' })

# 2. Partage SMB
Write-Host "`n2. Partage SMB :" -ForegroundColor Yellow
$share = Get-SmbShare -Name "ClusterWitness" -ErrorAction SilentlyContinue
if ($share) {
    Write-Host "   Nom    : $($share.Name)" -ForegroundColor Green
    Write-Host "   Chemin : $($share.Path)" -ForegroundColor Green
} else {
    Write-Host "   ABSENT" -ForegroundColor Red
}

# 3. Droits SMB
Write-Host "`n3. Droits SMB :" -ForegroundColor Yellow
Get-SmbShareAccess -Name "ClusterWitness" -ErrorAction SilentlyContinue |
    Format-Table AccountName, AccessRight, AccessControlType -AutoSize

# 4. Droits NTFS
Write-Host "4. Droits NTFS (filtré sur CL-SQL-AG) :" -ForegroundColor Yellow
Get-Acl "P:\ClusterWitness" | Select-Object -ExpandProperty Access |
    Where-Object { $_.IdentityReference -like "*CL-SQL-AG*" } |
    Format-Table IdentityReference, FileSystemRights, AccessControlType -AutoSize


# 5. Test d'accès depuis SQLS01
Write-Host "`n5. Test d'accès depuis SQLS01 :" -ForegroundColor Yellow
Write-Host "   (à exécuter sur SQLS01)" -ForegroundColor Gray
Write-Host "   Test-Path '\\OPT-FS03\\ClusterWitness'" -ForegroundColor Gray
```

Test d'accès depuis SQLS01

```
# Sur SQLS01
Test-Path "\\OPT-FS03\ClusterWitness"
# Doit retourner : True

# Test d'écriture
$testFile = "\\OPT-FS03\ClusterWitness\test_$(Get-Date -Format 'yyyyMMddHHmmss').txt"
"Test" | Out-File -FilePath $testFile
Test-Path $testFile

# Test de supression
Remove-Item $testFile -Force
```

```
PS C:\Users\Administrateur> Test-Path "\\OPT-FS03\ClusterWitness"
True

PS C:\Users\Administrateur> $testFile = "\\OPT-FS03\ClusterWitness\test_$(Get-Date -Format 'yyyyMMddHHmmss').txt"

PS C:\Users\Administrateur> "Test" | Out-File -FilePath $testFile

PS C:\Users\Administrateur> Test-Path $testFile
True

PS C:\Users\Administrateur> HOSTNAME.EXE
SQLS01
```

4.4 Quorum

> **Prérequis :** *Le partage \\OPT-FS03\ClusterWitness doit être créé et accessible en écriture par le compte du cluster. Créer le dossier **P:\ClusterWitness\ sur OPT-FS03**, le partager en ClusterWitness, et accorder les droits au compte ordinateur **CL-SQL-AG$.***

- **File Share Witness** sur OPT-FS03 (à default, Cloud Witness). Quorum (File Share Witness) sert à décider quel nœud est actif.

Script fourni : `4.4.0.configure_cluster_quorum.ps1` — *exécuté sur : SQLS01 **ou** SQLS02*

```
# Configurer le quorum avec le témoin de partage de fichiers
Set-ClusterQuorum -FileShareWitness "\\OPT-FS03\ClusterWitness"

# Vérifier la configuration
Get-ClusterQuorum

# Vérifier l'état détaillé
Get-ClusterResource | Where-Object { $_.ResourceType -eq "File Share Witness" } | Format-
List *
```

```
PS C:\Scripts\_Projet AlwaysOn> .\4.4.0.configure_cluster_quorum.ps1

Cluster              QuorumResource
-------              --------------
CL-SQL-AG            Témoin de partage de fichiers
Quorum configure (File Share Witness sur OPT-FS03).


PS C:\Scripts\_Projet AlwaysOn> HOSTNAME.EXE
SQLS01
PS C:\Scripts\_Projet AlwaysOn> Get-ClusterQuorum

Cluster              QuorumResource
-------              --------------
CL-SQL-AG            Témoin de partage de fichiers


PS C:\Scripts\_Projet AlwaysOn> Get-ClusterResource | Where-Object { $_.ResourceType -eq "File Share Witness" } | Format-List *


Characteristics         : Quorum
Cluster                 : CL-SQL-AG
DeadlockTimeout         : 300000
Description             :
Id                      : 053e830a-636a-455a-8f80-f5e01c65fccc
IsCoreResource          : True
EmbeddedFailureAction   : 2
IsAlivePollInterval     : 4294967295
IsNetworkClassResource  : False
IsStorageClassResource  : False
LastOperationStatusCode : 0
LooksAlivePollInterval  : 4294967295
MaintenanceMode         : False
MonitorProcessId        : 9528
Name                    : Témoin de partage de fichiers
OwnerGroup              : Groupe du cluster
OwnerNode               : SQLS01
PendingTimeout          : 180000
PersistentState         : 1
ResourceSpecificData1   : 0
ResourceSpecificData2   : 0
ResourceSpecificStatus  :
ResourceType            : File Share Witness
RestartAction           : 2
RestartDelay            : 500
RestartPeriod           : 600000
RestartThreshold        : 1
RetryPeriodOnFailure    : 600000
SeparateMonitor         : False
State                   : Online
StatusInformation       : 0
```

⚠️ le rapport de validation indiquait avant : *"Le cluster n'est pas configuré avec un témoin de quorum. Il est recommandé de configurer un témoin de quorum pour obtenir la plus haute disponibilité possible du cluster."*

Cet avertissement est maintenant résolu. Si on réexécute la validation (`4.2.0.validate_cluster.ps1`), on verra que l'avertissement de quorum a disparu.

Vérification : Le Quorum est-il Bien Fonctionnel ?

Faisons quelques tests pour confirmer que tout fonctionne.

Test 1 : Vérifier la configuration complète du quorum

powershell
```
# Voir les détails du quorum
Get-ClusterQuorum | Format-List *

# Voir l'état du vote des nœuds
Get-ClusterNode | Select-Object Name, State, NodeWeight, DynamicWeight | Format-Table -AutoSize
```

Test 2 : Vérifier que le témoin est bien accessible

powershell
```
# Depuis SQLS01 ou SQLS02, tester l'accès au partage témoin
Test-Path "\\OPT-FS03\ClusterWitness"
# Doit retourner : True

# Vérifier que le cluster peut écrire dans le témoin
$witness = (Get-ClusterQuorum).QuorumResource
Write-Host "Ressource témoin : $witness"
```

Test 3 : Vérifier les logs du cluster

powershell
```
# Voir les derniers événements du cluster
Get-ClusterLog -TimeSpan 5 -Destination "C:\Temp\ClusterLogs"
# Ouvre ensuite C:\Temp\ClusterLogs\*_cluster.log
```

Test 4 : Simuler la perte d'un nœud (optionnel, en lab uniquement)

⚠️ **À faire uniquement si vous êtes en environnement de test.** Cette manipulation va mettre un nœud en pause et vérifier que le cluster reste en ligne grâce au témoin.

powershell
```
# Sur SQLS02 (le nœud secondaire)
Stop-ClusterNode -Name "SQLS02"

# Depuis SQLS01, vérifier que le cluster est toujours en ligne
Get-ClusterNode
# SQLS01 doit être "Up", SQLS02 doit être "Down"
# Le cluster doit rester fonctionnel
```

```
# Redémarrer SQLS02
Start-ClusterNode -Name "SQLS02"

# Vérifier que tout est revenu
Get-ClusterNode
# SQLS01 et SQLS02 doivent être "Up"
```

Résolution de l'Avertissement Restant (AD Permissions)

Rappelez-vous, il reste **un avertissement** dans le rapport de validation :

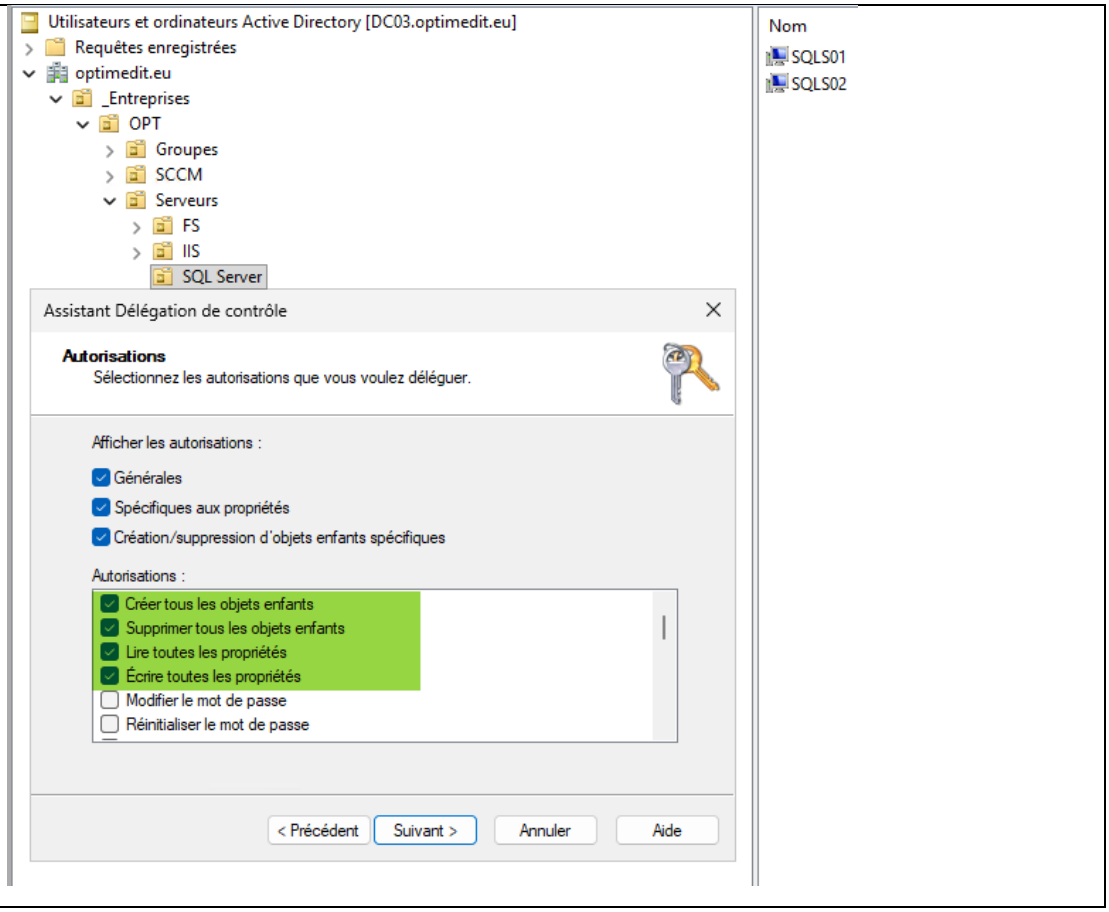
⚠️ *"Le nom réseau du cluster CL-SQL-AG ne dispose pas d'autorisations de création d'objets d'ordinateur sur l'unité d'organisation OU=SQL Server,OU=Serveurs,OU=OPT,OU=_Entreprises,DC=optimedit,DC=eu. Cela peut donner lieu à des problèmes lors de la création de noms réseau supplémentaires dans cette unité d'organisation."*

Cet avertissement est **important** car il affectera la création du **Listener Always On** (`SQL-AG-LSTNR`) à l'étape 5.5. Il faut donc le résoudre avant de continuer.

Solution : Déléguer les droits sur l'OU `SQL Server`

Sur `DC03`, dans "Utilisateurs et ordinateurs Active Directory" :

1. Activez "**Fonctionnalités avancées**" (menu `Affichage`).
2. Naviguez jusqu'à l'OU `OU=SQL Server,OU=Serveurs,OU=OPT,OU=_Entreprises,DC=optimedit,DC=eu`.
3. Faites un **clic droit** sur cette OU → "**Déléguer le contrôle...**".
4. Suivez l'assistant :
   - **Utilisateurs ou groupes** : `CL-SQL-AG$` (le compte du cluster)
   - **Cocher** tout en bas: Créer une tâche personnalisée à déléguer
   - **Tâches à déléguer** : "Créer un objet personnalisé dans ce dossier..." → "Objets Ordinateur"
   - **Autorisations** : Cocher "Spécifiques aux propriétés" et "Création/Suppression d'objets enfants spécifiques" (ou au minimum "Création"), puis cocher les 4 soulignés en verts
5. Terminez l'assistant.



Alternative en PowerShell (sur DC03) :

```powershell
# Importer le module ActiveDirectory
Import-Module ActiveDirectory

# Paramètres
$OU = "OU=SQL Server,OU=Serveurs,OU=OPT,OU=_Entreprises,DC=optimedit,DC=eu"
$ClusterAccount = "CL-SQL-AG$"

# Récupérer les ACL de l'OU
$acl = Get-Acl "AD:$OU"

# Créer la règle de délégation
$identity = New-Object System.Security.Principal.NTAccount("OPTIMEDIT", $ClusterAccount)
$adRights = [System.DirectoryServices.ActiveDirectoryRights]::CreateChild -bor
[System.DirectoryServices.ActiveDirectoryRights]::DeleteChild
$inheritanceType = [System.DirectoryServices.ActiveDirectorySecurityInheritance]::All
$objectType = [System.Guid]"bf967a86-0de6-11d0-a285-00aa003049e2"  # GUID de la classe
Computer

$rule = New-Object System.DirectoryServices.ActiveDirectoryAccessRule(
    $identity,
    $adRights,
    "Allow",
    $objectType,
    $inheritanceType
)

# Appliquer la règle
$acl.AddAccessRule($rule)
Set-Acl -Path "AD:$OU" -AclObject $acl

Write-Host "Droits délégués sur l'OU : $OU" -ForegroundColor Green
Write-Host "Compte : $ClusterAccount" -ForegroundColor Green
```

Vérification Post-Délégation (sur le DC03)

```powershell
# Variables
Import-Module ActiveDirectory

$OU = "OU=SQL Server,OU=Serveurs,OU=OPT,OU=_Entreprises,DC=optimedit,DC=eu"

# Voir les ACE sur l'OU (filtré sur CL-SQL-AG)
$acl = Get-Acl "AD:$OU"
$acl.Access | Where-Object {
    $_.IdentityReference -like "*CL-SQL-AG*"
} | Format-List IdentityReference, ActiveDirectoryRights, ObjectType, InheritanceType
```

Résultat attendu : Vous devriez voir une entrée pour `OPTIMEDIT\CL-SQL-AG$` avec :

- `ActiveDirectoryRights` : `CreateChild, DeleteChild, ReadProperty, ...`
- `ObjectType` : `bf967a86-0de6-11d0-a285-00aa003049e2` (GUID de la classe Computer)
- `InheritanceType` : `All` ou `Descendents`

```powershell
# Variables
$OU = "OU=SQL Server,OU=Serveurs,OU=OPT,OU=_Entreprises,DC=optimedit,DC=eu"

# Voir les ACE sur l'OU (filtré sur CL-SQL-AG)
$acl = Get-Acl "AD:$OU"
$acl.Access | Where-Object {
    $_.IdentityReference -like "*CL-SQL-AG*"
} | Format-List IdentityReference, ActiveDirectoryRights, ObjectType, InheritanceType
```

```
IdentityReference     : OPTIMEDIT\CL-SQL-AG$
ActiveDirectoryRights : CreateChild, DeleteChild, ReadProperty, WriteProperty
ObjectType            : 00000000-0000-0000-0000-000000000000
InheritanceType       : Descendents

IdentityReference     : OPTIMEDIT\CL-SQL-AG$
ActiveDirectoryRights : CreateChild
ObjectType            : bf967a86-0de6-11d0-a285-00aa003049e2
InheritanceType       : All

IdentityReference     : OPTIMEDIT\CL-SQL-AG$
ActiveDirectoryRights : ReadProperty, WriteProperty
ObjectType            : fa4693bb-7bc2-4cb9-81a8-c99c43b7905e
InheritanceType       : Descendents

IdentityReference     : OPTIMEDIT\CL-SQL-AG$
ActiveDirectoryRights : CreateChild, DeleteChild
ObjectType            : ddc790ac-af4d-442a-8f0f-a1d4caa7dd92
InheritanceType       : Descendents
```

Test définitif (la seule qui compte vraiment)

La **seule méthode vraiment fiable** est le test réel : créer le Listener Always On.

Voici le test que vous pouvez faire **dès maintenant** (avant même d'avoir créé l'AG) :

Test : Créer un objet ordinateur avec les droits du CNO via une tâche planifiée

Sur `SQLS01`, en tant qu'administrateur :

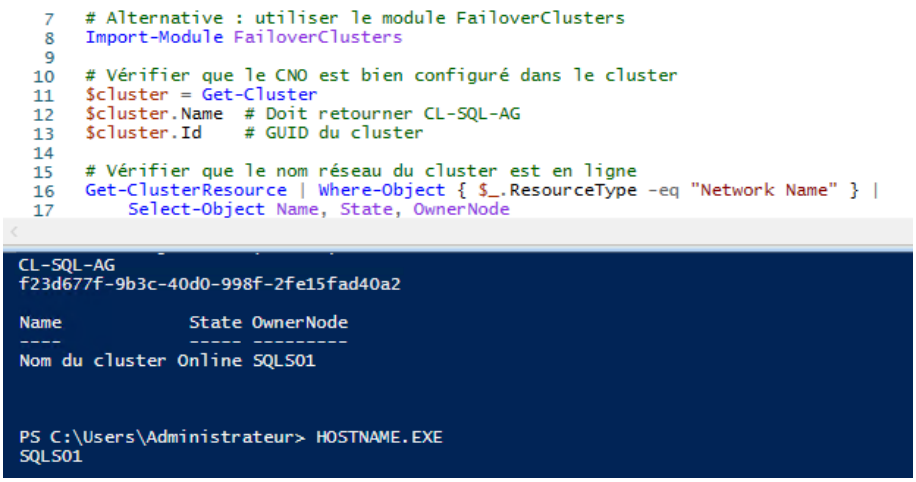
```powershell
# Créer une tâche planifiée qui s'exécute sous le compte CL-SQL-AG$
# (le mot de passe est récupéré depuis la base du cluster)

# 1. Obtenir le mot de passe du CNO (uniquement possible sur un nœud de cluster)
# Cette commande est complexe et rarement utilisée en pratique.
```

```powershell
# Alternative : utiliser le module FailoverClusters
Import-Module FailoverClusters

# Vérifier que le CNO est bien configuré dans le cluster
$cluster = Get-Cluster
$cluster.Name  # Doit retourner CL-SQL-AG
$cluster.Id    # GUID du cluster

# Vérifier que le nom réseau du cluster est en ligne
Get-ClusterResource | Where-Object { $_.ResourceType -eq "Network Name" } |
    Select-Object Name, State, OwnerNode
```



Vérification Complémentaire Utile

```powershell
# Vérifier que le CNO est bien authentifié dans AD
Import-Module FailoverClusters

# Lister les ressources du cluster avec leur état
Get-ClusterResource | Select-Object Name, ResourceType, State, OwnerNode |
    Format-Table -AutoSize

# Vérifier spécifiquement le nom réseau
Get-ClusterResource -Name "Nom du cluster" | Format-List *
```

Récapitulatif : Toutes les Vérifications sont au Vert

Vérification	Résultat	Statut
Cluster CL-SQL-AG créé	Name = CL-SQL-AG	✅
Adresse IP du cluster	192.168.1.253 en ligne	✅
CNO authentifié dans AD	Nom réseau Online	✅
DNS enregistré	CL-SQL-AG.optimedit.eu résout	✅
SPN du cluster	MSServerCluster/... enregistrés	✅
Témoin de quorum	Online sur \\OPT-FS03\ClusterWitness	✅

Vérification	Résultat	Statut
Délégation AD sur OU	CreateChild + Computer GUID	✅
Quorum configuré	Majority avec File Share Witness	✅

5. Phase 3 — Always On SQL Server

5.0.0 Creation de SPN

5.0.0.manage_sql_spn.ps1

```
# Verification si y a pas de doublons de SPN (sur DC03 ou les noeuds):
# Vérifier si MSSQLSvc/SQLS01.optimedit.eu:1433 apparaît à la fois sous SVC_SQL_Engine et sous SQLS01

setspn -X
```

Résultat

```
0 groupe de SPN en double détectés.
```

```
# Vérifie spécifiquement que les comptes ordinateur n'ont pas hérité les mêmes SPN
setspn -L SQLS01$
setspn -L SQLS02$
```

```
1  # Vérifie spécifiquement que les comptes ordinateur n'ont pas hérité les mêmes SPN
2  setspn -L SQLS01$
3  setspn -L SQLS02$
4  setspn -L SVC_SQL_Engine
```

```
PS C:\Users\Administrateur.OPTIMEDIT> # Vérifie spécifiquement que les comptes ordinateur n'ont pas hérité les mêmes SPN
setspn -L SQLS01$
setspn -L SQLS02$
setspn -L SVC_SQL_Engine
Noms ServicePrincipalName inscrits pour CN=SQLS01,OU=SQL Server,OU=Serveurs,OU=OPT,OU=_Entreprises,DC=optimedit,DC=eu:
    MSServerClusterMgmtAPI/SQLS01
    MSServerClusterMgmtAPI/SQLS01.optimedit.eu
    TERMSRV/SQLS01
    TERMSRV/SQLS01.optimedit.eu
    WSMAN/SQLS01
    WSMAN/SQLS01.optimedit.eu
    RestrictedKrbHost/SQLS01
    HOST/SQLS01
    RestrictedKrbHost/SQLS01.optimedit.eu
    HOST/SQLS01.optimedit.eu
Noms ServicePrincipalName inscrits pour CN=SQLS02,OU=SQL Server,OU=Serveurs,OU=OPT,OU=_Entreprises,DC=optimedit,DC=eu:
    MSServerClusterMgmtAPI/SQLS02
    MSServerClusterMgmtAPI/SQLS02.optimedit.eu
    WSMAN/SQLS02
    WSMAN/SQLS02.optimedit.eu
    TERMSRV/SQLS02
    TERMSRV/SQLS02.optimedit.eu
    RestrictedKrbHost/SQLS02
    HOST/SQLS02
    RestrictedKrbHost/SQLS02.optimedit.eu
    HOST/SQLS02.optimedit.eu
Noms ServicePrincipalName inscrits pour CN=SVC_SQL_Engine,OU=Service Accounts,DC=optimedit,DC=eu:
    MSSQLSvc/SQLS02
    MSSQLSvc/SQLS02:1433
    MSSQLSvc/SQLS02.optimedit.eu
    MSSQLSvc/SQLS02.optimedit.eu:1433
    MSSQLSvc/SQLS01
    MSSQLSvc/SQLS01:1433
    MSSQLSvc/SQLS01.optimedit.eu
    MSSQLSvc/SQLS01.optimedit.eu:1433

PS C:\Users\Administrateur.OPTIMEDIT> HOSTNAME.EXE
SQLS02
```

Élément	Description / Rôle
SVC_SQL_Engine	Le compte de service (une identité dans AD).
MSSQLSvc/...	Le Service Principal Name (SPN) — le nom du service.

Le lien	Le SPN **MSSQLSvc**/SQLS01.optimedit.eu:1433 est enregistré sur le compte **SVC_SQL_Engine**.

```
# Vérifier que le SPN est bien enregistré et résolu
setspn -L optimedit\SVC_SQL_Engine
```

Résultat : Il devrait avoir 8 SPN pour le compte SVC_SQL_Engine (4 variantes, sur les deux instances)

```
setspn -L optimedit\SVC_SQL_Engine
```

```
PS C:\Users\Administrateur.OPTIMEDIT> setspn -L optimedit\SVC_SQL_Engine
Noms ServicePrincipalName inscrits pour CN=SVC_SQL_Engine,OU=Service Accounts,DC=optimedit,DC=eu:
    MSSQLSvc/SQLS02
    MSSQLSvc/SQLS02:1433
    MSSQLSvc/SQLS02.optimedit.eu
    MSSQLSvc/SQLS02.optimedit.eu:1433
    MSSQLSvc/SQLS01
    MSSQLSvc/SQLS01:1433
    MSSQLSvc/SQLS01.optimedit.eu
    MSSQLSvc/SQLS01.optimedit.eu:1433
```

5.0.1 Verification si authentication Kerberos ou NTLM

Sur SQLS02 en tant qu'admin de domaine:

```
# 1. Purger le cache
klist purge

# 2. Se connecter à SQLS01
sqlcmd -S SQLS01.optimedit.eu -E -Q "SELECT @@SERVERNAME"

# 3. Voir les tickets Kerberos obtenus
klist


# Sur SQLS02
klist purge
sqlcmd -S SQLS01.optimedit.eu -E -Q "SELECT auth_scheme FROM sys.dm_exec_connections WHERE session_id = @@SPID"
```

Résultat attendu : KERBEROS, si NTLM vérifier si ce n'est pas bon

```
sqlcmd -S SQLS01.optimedit.eu -E -Q "SELECT auth_scheme FROM sys.dm_exec_connections WHERE session_id = @@SPID"
```

```
PS C:\Users\Administrateur.OPTIMEDIT> sqlcmd -S SQLS01.optimedit.eu -E -Q "SELECT auth_scheme FROM sys.dm_exec_connections WHERE session_id = @@SPID"
auth_scheme
------------------------------------
KERBEROS

(1 lignes affect,es)

PS C:\Users\Administrateur.OPTIMEDIT> HOSTNAME.EXE
SQLS02
```

Sur OPT-FS03 en tant qu'admin de domaine

```
# Test de connexion SQL via PowerShell
$connectionString = "Server=SQLS01.optimedit.eu;Database=master;Integrated Security=True;Encrypt=True;TrustServerCertificate=True"
$connection = New-Object System.Data.SqlClient.SqlConnection($connectionString)
$connection.Open()

$command = $connection.CreateCommand()
$command.CommandText = "SELECT auth_scheme FROM sys.dm_exec_connections WHERE session_id = @@SPID"
$result = $command.ExecuteScalar()
```

```
Write-Host "Auth scheme : $result" -ForegroundColor Cyan
$connection.Close()
```

Résultat attendu : KERBEROS, si NTLM ce n'est pas bon

```
# Test de connexion SQL via PowerShell
$connectionString = "Server=SQLS01.optimedit.eu;Database=master;Integrated Security=True;Encrypt=True;TrustServerCertificate=True"
$connection = New-Object System.Data.SqlClient.SqlConnection($connectionString)
$connection.Open()

$command = $connection.CreateCommand()
$command.CommandText = "SELECT auth_scheme FROM sys.dm_exec_connections WHERE session_id = @@SPID"
$result = $command.ExecuteScalar()

Write-Host "Auth scheme : $result" -ForegroundColor Cyan
$connection.Close()
```

```
PS C:\Users\Administrateur.OPTIMEDIT> # Test de connexion SQL via PowerShell
$connectionString = "Server=SQLS01.optimedit.eu;Database=master;Integrated Security=True;Encrypt=True;TrustServerCertificate=True"
$connection = New-Object System.Data.SqlClient.SqlConnection($connectionString)
$connection.Open()

$command = $connection.CreateCommand()
$command.CommandText = "SELECT auth_scheme FROM sys.dm_exec_connections WHERE session_id = @@SPID"
$result = $command.ExecuteScalar()

Write-Host "Auth scheme : $result" -ForegroundColor Cyan
$connection.Close()
Auth scheme : KERBEROS

PS C:\Users\Administrateur.OPTIMEDIT> HOSTNAME.EXE
OPT-FS03
```

5.0.2 Activation des logs Audit Kerberos

🎯 Activer l'audit Kerberos (similaire à NTLM)

L'audit Kerberos ne se configure pas au même endroit que l'audit NTLM. Il se trouve dans la **Configuration avancée des stratégies d'audit**.

Sur votre contrôleur de domaine (DC03), via la gestion des stratégies de groupe (`gpmc.msc`) :

1. **Créer ou éditer un GPO** : Créez un nouveau GPO (par exemple « Audit Kerberos ») ou éditez le `Default Domain Controllers Policy`.
2. **Naviguer dans l'arborescence** : Allez dans `Configuration ordinateur` → `Stratégies` → `Paramètres Windows` → `Paramètres de sécurité` → `Configuration avancée des stratégies d'audit` → `Stratégies d'audit` → **`Ouverture de session de compte`**.
3. **Activer les deux stratégies** : Vous verrez deux entrées Kerberos dans le volet de droite. Double-cliquez sur chacune et cochez **`Réussite`** et **`Échec`** :
   - **Auditer le service d'authentification Kerberos** (génère l'événement 4768 pour les demandes de TGT)
   - **Auditer les opérations de ticket de service Kerberos** (génère l'événement 4769 pour les demandes de TGS)
4. **Appliquer** : Après avoir validé, forcez la mise à jour de la stratégie sur le DC avec la commande `gpupdate /force`.

📊 Où consulter les événements Kerberos ?

Une fois l'audit activé, les événements apparaîtront dans l'**Observateur d'événements** de votre contrôleur de domaine, dans `Journaux Windows` → **`Sécurité`**.

Vous pourrez filtrer par ID d'événement :

- **4768** : Demande de ticket TGT (authentification initiale)

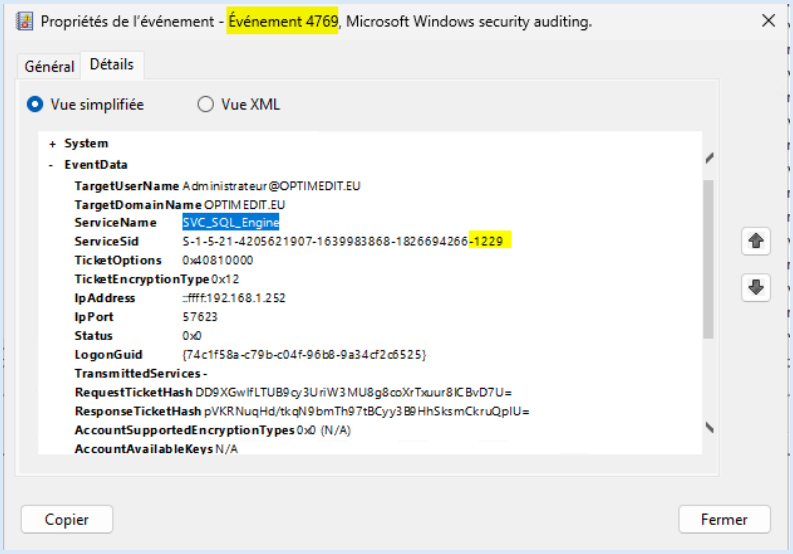
- **4769** : Demande de ticket de service (accès à une ressource comme SQL Server)
- **4771** : Échec de pré-authentification Kerberos

```
#Sur SQLS02 en tant qu'admin de domaine
# 1. Purger le cache
klist purge

# 2. Se connecter à SQLS01
sqlcmd -S SQLS01.optimedit.eu -E -Q "SELECT @@SERVERNAME"

# 3. Voir les tickets Kerberos obtenus
klist
```

Journaux Windows → **Sécurité -ID 4768 - 4769)**



5.1 Activer Always On

Script fourni : `5.1.0.enable_alwayson_feature.ps1` — *exécuté sur : SQLS01 + SQLS02*

```
Enable-SqlAlwaysOn -ServerInstance $env:COMPUTERNAME -Force
Restart-Service -Name "MSSQLSERVER" -Force
```

5.2 Créer les endpoints Always On

Script fourni : `5.2.0.create_hadr_endpoint.sql` — *exécuté sur : SQLS01 + SQLS02*

```
CREATE ENDPOINT [Hadr_endpoint] STATE = STARTED
AS TCP (LISTENER_PORT = 5022)
FOR DATABASE_MIRRORING (ROLE = ALL, AUTHENTICATION = WINDOWS NEGOTIATE,
    ENCRYPTION = REQUIRED ALGORITHM AES);
GO
GRANT CONNECT ON ENDPOINT::[Hadr_endpoint] TO [optimedit\SVC_SQL_Engine];
```

5.3 Créer l'Availability Group (sur SQLS01, primaire)

Script fourni : `5.3.0.create_availability_group.sql` — *exécuté sur : SQLS01 (Primary)*

```
CREATE AVAILABILITY GROUP [AG-OPTIMEDIT]
FOR REPLICA ON
N'SQLS01' WITH (ENDPOINT_URL = 'TCP://192.168.1.251:5022',
```

```
    AVAILABILITY_MODE = SYNCHRONOUS_COMMIT, FAILOVER_MODE = AUTOMATIC, SEEDING_MODE =
AUTOMATIC),
N'SQLS02' WITH (ENDPOINT_URL = 'TCP://192.168.1.252:5022',
    AVAILABILITY_MODE = SYNCHRONOUS_COMMIT, FAILOVER_MODE = AUTOMATIC, SEEDING_MODE =
AUTOMATIC);
```

5.4 Rejoindre l'AG (sur SQLS02, secondaire)

Script fourni : `5.4.0.join_availability_group_secondary.sql` — *exécuté sur : SQLS02 (Secondary)*

```
ALTER AVAILABILITY GROUP [AG-OPTIMEDIT] JOIN;
GO
ALTER AVAILABILITY GROUP [AG-OPTIMEDIT] GRANT CREATE ANY DATABASE;
```

5.5 Créer le Listener

Script fourni : `5.5.0.create_ag_listener.sql` — *exécuté sur : SQLS01 (Primary)*

```
ALTER AVAILABILITY GROUP [AG-OPTIMEDIT]
ADD LISTENER 'SQL-AG-LSTNR'
(WITH IP (('192.168.1.250','255.255.255.0')), PORT = 1433);
```

Remarque : *vérifier que SQL-AG-LSTNR et CL-SQL-AG résolvent en DNS*

5.6 Tester le failover

Script fourni : `5.6.0.test_failover.ps1` — *exécuté sur : SQLS02*

```
Switch-SqlAvailabilityGroup -Path "SQLSERVER:\SQL\SQLS02\DEFAULT\AvailabilityGroups\AG-
OPTIMEDIT"
```

6. Phase 4 — Création des bases de données

Les 13 bases applicatives : DB_Direction, DB_Comptabilite, DB_Paie, DB_RH, DB_CE, DB_IT, DB_Production, DB_Formation, DB_Achat, DB_Commercial, DB_Client, DB_Juridique, DB_Blog.

6.1 Création des 13 bases (sur SQLS01)

Script fourni : `6.1.0.create_application_databases.sql` — *exécuté sur : SQLS01 (Primary)*

```
-- Boucle idempotente sur les 13 bases :
-- CREATE DATABASE ... COLLATE SQL_Latin1_General_CP1_CI_AS
-- ALTER DATABASE ... SET RECOVERY FULL
-- BACKUP DATABASE / BACKUP LOG
-- (script complet dans l'archive)
```

6.2 Ajout des bases à l'AG

Script fourni : `6.2.0.add_databases_to_ag.sql` — *exécuté sur : SQLS01 (Primary)*

```
ALTER AVAILABILITY GROUP [AG-OPTIMEDIT]
ADD DATABASE DB_Direction, DB_Comptabilite, DB_Paie, DB_RH, DB_CE,
DB_IT, DB_Production, DB_Formation, DB_Achat, DB_Commercial,
DB_Client, DB_Juridique, DB_Blog;
```

Remarque : *Le seeding automatique (activé à l'étape 5.3) synchronise ces bases vers SQLS02 sans restauration manuelle. Vérifier l'état avec **sys.dm_hadr_database_replica_states** (chapitre 11).*

7. Phase 5 — Sécurisation et maintenance

7.1 Transparent Data Encryption (TDE)

Script fourni : `7.1.0.enable_tde.ps1` — *exécuté sur : **SQLS01** (Primary)*

```
..
$key = Get-Content "$SecureFolder\aes.key" -Encoding Byte

$secureMaster = Get-Content "$SecureFolder\tde-masterkey-pwd.enc" | ConvertTo-
SecureString -Key $key

$secureCert   = Get-Content "$SecureFolder\tde-cert-pwd.enc"      | ConvertTo-
SecureString -Key $key

# CREATE MASTER KEY + CREATE CERTIFICATE + BACKUP CERTIFICATE via Invoke-Sqlcmd

# puis boucle sur les 13 bases : CREATE DATABASE ENCRYPTION KEY + ALTER DATABASE SET
ENCRYPTION ON
..
```

Important : *Sans l'étape 7.1.1 ci-dessous, une bascule vers SQLS02 rend les bases chiffrées illisibles : le secondaire ne peut pas déchiffrer la clé de chiffrement de base (DEK) sans le certificat.*

7.1.1 Restaurer le certificat TDE sur le secondaire

Copier au préalable TDE_Cert.cer et TDE_Cert.pvk depuis S:\SQL\Backup (SQLS01) vers un emplacement local sur SQLS02, hors de l'AG (ex. C:\Certs\).

Script fourni : `7.1.1.restore_tde_cert_on_secondary.ps1` — *exécuté sur : SQLS02 (Secondary)*

```
..
$key = Get-Content "$SecureFolder\aes.key" -Encoding Byte

$securePwd = Get-Content "$SecureFolder\tde-cert-pwd.enc" | ConvertTo-SecureString -Key
$key

# CREATE CERTIFICATE TDE_Cert FROM FILE ... WITH PRIVATE KEY (... DECRYPTION BY PASSWORD
= $plainPwd)

# via Invoke-Sqlcmd sur SQLS02
..
```

7.2 Jobs SQL Server Agent (Primary)

Script fourni : `7.2.0.create_maintenance_jobs.sql` — *exécuté sur : SQLS01 (Primary)*

```
-- Jobs : Backup FULL hebdomadaire, DBCC CHECKDB hebdomadaire
-- (exemple simplifié – pour la production, envisager Ola Hallengren
--  MaintenanceSolution.sql)
```

Important : *msdb n'est pas répliquée par l'AG : sans l'étape 7.2.1, plus aucun job ne s'exécute après une bascule vers SQLS02.*

7.2.1 Synchroniser les jobs sur le secondaire

Script fourni : `7.2.1.sync_maintenance_jobs_secondary.sql` — *exécuté sur : SQLS02 (Secondary)*

```
-- Mêmes jobs que 7.2.0, avec garde AG-aware :
-- IF (SELECT sys.fn_hadr_is_primary_replica('DB_Direction')) = 1
-- BEGIN
--     BACKUP DATABASE [DB_Direction] TO DISK = 'S:\SQL\Backup\DB_Direction_FULL.bak';
-- END
-- (le job existe sur les deux nœuds mais ne s'exécute que sur le primaire du moment)
```

7.3 Alertes SQL Server

Script fourni : `7.3.0.create_alerts.sql` — *exécuté sur : SQLS01 + SQLS02*

```
-- sp_add_alert pour les sévérités 19 à 25
-- (configurer Database Mail + un opérateur pour recevoir les notifications)
```

7.4 Audits SQL Server

Script fourni : `7.4.0.enable_audit.sql` — *exécuté sur : SQLS01 + SQLS02*

```
CREATE SERVER AUDIT [Audit-OPTIMEDIT] TO FILE (FILEPATH = 'S:\SQL\Backup\Audit\');
ALTER SERVER AUDIT [Audit-OPTIMEDIT] WITH (STATE = ON);
-- + spécifications : connexions, changements de schéma, élévations de privilèges
```

Remarque : *Le dossier S:\SQL\Backup\Audit\ est désormais pré-créé par le script 3.1.0 (section 3.1).*

7.5 Optimisation TempDB

TempDB n'est pas répliquée par Always On ; à appliquer identiquement sur SQLS01 et SQLS02.

Script fourni : `7.5.0.optimize_tempdb.sql` — *exécuté sur : SQLS01 + SQLS02*

```
ALTER DATABASE tempdb MODIFY FILE (NAME = tempdev, SIZE = 2GB, FILEGROWTH = 512MB);
ALTER DATABASE tempdb ADD FILE (NAME = tempdev2, FILENAME = 'S:\SQL\TempDB\tempdb2.ndf',
SIZE = 2GB, FILEGROWTH = 512MB);
ALTER DATABASE tempdb ADD FILE (NAME = tempdev3, FILENAME = 'S:\SQL\TempDB\tempdb3.ndf',
SIZE = 2GB, FILEGROWTH = 512MB);
ALTER DATABASE tempdb ADD FILE (NAME = tempdev4, FILENAME = 'S:\SQL\TempDB\tempdb4.ndf',
SIZE = 2GB, FILEGROWTH = 512MB);
```

8. Phase 6 — Intégration IIS (sessions SQL Server)

8.1 Comptes de service applicatifs (13 comptes svc_*)

Script fourni : `8.1.0.create_iis_service_accounts.ps1` — *exécuté sur : DC03*

```
$Accounts = @("svc_direction","svc_comptabilite","svc_paie","svc_rh","svc_ce",
  "svc_it","svc_production","svc_formation","svc_achat","svc_commercial",
  "svc_client","svc_juridique","svc_blog")
# boucle New-ADUser (voir script complet dans l'archive)
```

8.2 Groupe Active Directory GG_IIS_AppPools

Vérifie que les 13 comptes svc_* existent, crée le groupe s'il n'existe pas, puis y ajoute les 13 comptes.

Script fourni : `8.2.0.create_GG_SVC_AppPools.ps1` — *exécuté sur : DC03*

```
# 1. Verifie l'existence des 13 comptes svc_*
# 2. New-ADGroup -Name "GG_IIS_AppPools" -GroupScope Global -GroupCategory Security
# 3. Add-ADGroupMember -Identity "GG_IIS_AppPools" -Members <les 13 comptes>
# (script complet et idempotent dans l'archive)
```

8.3 Base de sessions ASPState (avant le login — ordre corrigé)

Important : *Cette étape doit précéder la création du login SQL (8.4) : la boucle de droits de 8.4.0 référence ASPState par son nom — si elle n'existe pas encore, elle est silencieusement absente de la boucle et GG_IIS_AppPools n'obtient jamais de droits dessus.*

Script fourni : `8.3.0.create_aspstate_database.ps1` — *exécuté sur : SQLS01 (Primary)*

```
aspnet_regsql.exe -S SQL-AG-LSTNR -E -ssadd
```

Script fourni : `8.3.1.add_aspstate_to_ag.sql` — *exécuté sur : SQLS01 (Primary)*

```
ALTER AVAILABILITY GROUP [AG-OPTIMEDIT] ADD DATABASE ASPState;
```

8.4 Login SQL Server (après ASPState)

Script fourni : `8.4.0.create_sql_login_iis_group.sql` — *exécuté sur : SQLS01 (Primary)*

```
CREATE LOGIN [optimedit\GG_IIS_AppPools] FROM WINDOWS;
-- puis boucle sur les 14 bases (ASPState + 13 bases applicatives) :
-- CREATE USER ... FOR LOGIN ...
-- ALTER ROLE db_datareader / db_datawriter ADD MEMBER ...
-- le script verifie explicitement la presence d'ASPState avant de conclure
```

8.5 Configuration des 13 sites IIS

Script fourni : `8.5.0.configure_iis_sessionstate.ps1` — *exécuté sur : chaque serveur IIS*

```
# Parcourt tous les sites IIS locaux et injecte/actualise :
# <sessionState mode="SQLServer"
#   sqlConnectionString="Data Source=SQL-AG-LSTNR;Integrated Security=True;"
#   cookieless="false" timeout="20" />
```

8.6 Tests de validation

- Persistance de session sous charge normale.
- Redémarrage d'un pool d'applications IIS → session conservée.
- Basculement Always On (SQLS01 → SQLS02) → session toujours valide, ASPState accessible via le Listener.

9. Tâches d'administration récurrentes

9.1 Quotidien

- Surveiller l'état de synchronisation Always On (Synchronized / healthy).
- Surveiller l'exécution des jobs SQL Agent sur le nœud primaire du moment.
- Analyser les deadlocks et les requêtes lentes.

9.2 Hebdomadaire

- Fragmentation des index, croissance des bases, DBCC CHECKDB, validité des sauvegardes.

9.3 Mensuel

- Test de restauration complète, test de bascule Always On (avec vérification des jobs et du certificat TDE sur le nouveau primaire), mises à jour Windows/SQL, revue des permissions AD/SQL.

9.4 Sauvegardes de clés AES vers un serveur de fichier

9.4.1 Création du partage SQLSecrets$

Avant de pouvoir sauvegarder les secrets, le partage \\OPT-FS03\SQLSecrets$ doit exister.

Script fourni : `9.4.1.create_sqlsecrets_share.ps1` — *exécuté sur le serveur de fichier **OPT-FS03***

Prérequis
Le volume `P:` doit exister sur OPT-FS03.
Le groupe `Administrateurs du domaine` doit être résolvable.
Exécuter en administrateur local sur OPT-FS03.

Principe :
Crée le dossier P:\SQLSecrets\.
Le partage en SQLSecrets$ (partage caché).
Accorde les droits NTFS et SMB aux Administrateurs du domaine uniquement.

9.4.2 Sauvegarde des secrets vers OPT-FS03

Script fourni : `9.4.2.backup_secrets_to_fileserver.ps1` — *exécuté sur : chaque nœud SQL Server*

La section 3.0 met en place la gestion des secrets : la clé AES est générée localement sur chaque nœud SQL (SQLS01, SQLS02), et les secrets sont chiffrés dans des fichiers .enc qui restent eux aussi locaux. C'est une politique volontairement stricte : les secrets ne circulent pas, ils ne sont pas exposés sur un partage réseau.

Conséquence : si le disque local d'un nœud est perdu (panne matérielle, sinistre, réinstallation), la clé AES et les fichiers .enc sont perdus avec lui. Or sans la clé AES, les secrets sont irrécupérables. C'est le scénario catastrophe qu'il faut anticiper.

Cette section 9.4 répond à ce risque en documentant une sauvegarde manuelle et délibérée de la clé AES et des fichiers .enc vers le serveur de fichiers **OPT-FS03.optimedit.eu**. Ce n'est pas une synchronisation automatique, c'est un acte administratif : on sauvegarde après avoir généré ou modifié un secret.

```
# Sauvegarde standard (copie les fichiers absents ou modifiés)
.\9.4.2.backup_secrets_to_fileserver.ps1
# Forcer la copie même si le fichier existe déjà côté serveur de fichiers
.\9.4.2.backup_secrets_to_fileserver.ps1 -Force
# Spécifier un autre partage de sauvegarde
.\9.4.2.backup_secrets_to_fileserver.ps1 -BackupShare "\\OPT-FS03\AutrePartage$"
```

Important : Cette sauvegarde ne doit pas être automatisée. La clé AES ne doit pas se retrouver sur un partage réseau sans contrôle. Chaque exécution doit être un acte délibéré de l'administrateur, tracé dans le journal des opérations.

Attention : Le partage SQLSecrets$ contient la clé AES, qui permet de déchiffrer tous les secrets de l'infrastructure SQL. Une compromission de ce partage compromet l'ensemble des secrets. Les ACL doivent être vérifiées régulièrement, et l'accès doit être audité.

9.5 Sauvegarde des clés AES dans KeePass

La clé AES générée par *3.0.0.manage_secrets.ps1* est un secret critique : sans elle, les fichiers .enc sont irrécupérables. En complément de la sauvegarde vers OPT-FS03 (section 9.4), il est recommandé de conserver une copie dans un gestionnaire de mots de passe d'entreprise. **KeePass** (ou KeePassXC) est adapté à cet usage.

9.5.1 Procédure de sauvegarde

- ✓ **Sur le nœud SQL** (SQLS01 ou SQLS02), convertir la clé AES en Base64 :

```
[Convert]::ToBase64String((Get-Content "C:\ProgramData\SQLSecrets\aes.key" -Encoding Byte))
```

- ✓ **Ouvrir la base KeePass** partagée de l'équipe infrastructure.
- ✓ **Créer une entrée** :

- **Titre** : `SQLSecrets - Clé AES - SQLS01` (ou `SQLS02`)
- **Nom d'utilisateur** : laisser vide ou indiquer `SYSTEM`
- **Mot de passe** : laisser vide
- **Champ personnalisé** : `AESKeyBase64` = chaîne Base64 obtenue à l'étape 1
- **Notes** : `Clé AES 256 bits pour déchiffrement des secrets SQL. Générée le <date>. Ne pas modifier.`

- ✓ **Sauvegarder la base KeePass.**

9.5.2 Procédure de restauration

En cas de perte de la clé locale (perte du disque, réinstallation du nœud, sinistre) :

- ✓ Ouvrir la base KeePass**, copier la valeur du champ AESKeyBase64.**
- ✓ Sur le nœud SQL**, restaurer la clé :**

```
$b64 = Read-Host "Coller la valeur Base64"

[Convert]::FromBase64String($b64) | Set-Content "C:\ProgramData\SQLSecrets\aes.key" -Encoding Byte
```

- ✓ **Réappliquer les ACL** via 3.0.0.manage_secrets.ps1 -GenerateKey (qui détecte la présence de la clé et ne la régénère pas, mais réapplique les ACL).

9.5.3 Avertissements de sécurité

Attention : La base KeePass elle-même doit être sauvegardée et protégée par un mot de passe maître fort. Une base KeePass non sauvegardée est aussi dangereuse qu'une clé AES non sauvegardée.

Important : La clé AES ne doit jamais être stockée en clair dans un fichier texte, un dépôt Git, un email ou un ticket. Le champ Base64 dans KeePass est acceptable car KeePass chiffre l'ensemble de sa base avec le mot de passe maître. En dehors de KeePass et du script 9.4.2, aucune copie en clair ne doit exister.

10. Annexe — Choix de la collation

Collation retenue : SQL_Latin1_General_CP1_CI_AS — insensible à la casse, sensible aux accents, compatible IIS/.NET, ASPState et homogène sur l'ensemble des bases de l'AG (exigence Always On).

Collation à éviter	Raison
`French_CI_AS`	Compatibilité limitée avec les applications .NET
`SQL_Latin1_General_CP1_CS_AS`	Sensible à la casse : incompatible avec la logique applicative IIS
`Latin1_General_BIN`	Tri binaire, inadapté aux applications web

11. Supervision avancée d'Always On

11.1 DMV essentielles

```
SELECT * FROM sys.dm_hadr_availability_replica_states;
SELECT * FROM sys.dm_hadr_database_replica_states;
SELECT * FROM sys.dm_hadr_availability_group_states;
```

11.2 Compteurs Perfmon

- SQLServer:Availability Replica — Bytes Sent to Replica/sec, Flow Control Time.
- SQLServer:Database Replica — Log Send Queue, Redo Queue, Mirrored Write Transactions/sec.

11.3 Extended Events

- availability_replica_state_change, hadr_sync_commit_performance, error_reported (sévérité ≥ 16).

11.4 Health Detection

- Flexible Failover Policy activée, alerte sur synchronization_health = DEGRADED/NOT_HEALTHY.

12. Sauvegardes de bases dans un Availability Group

12.1 Préférence de sauvegarde

Backup Preference	Comportement
Prefer Secondary	Sauvegarde sur un secondaire si disponible (recommandé)
Secondary only	Uniquement sur un secondaire
Primary	Toujours sur le primaire
Any Replica	N'importe quel réplica, selon priorité numérique

```
ALTER AVAILABILITY GROUP [AG-OPTIMEDIT] SET (AUTOMATED_BACKUP_PREFERENCE = SECONDARY);
```

12.2 Job de sauvegarde AG-aware

```
IF (SELECT sys.fn_hadr_backup_is_preferred_replica('DB_Direction')) = 1
BEGIN
    BACKUP DATABASE DB_Direction TO DISK = 'S:\SQL\Backup\DB_Direction.bak';
END
```

12.3 Stratégie de sauvegarde complete

La stratégie de sauvegarde des 13 bases de l'Availability Group AG-OPTIMEDIT repose sur **deux niveaux complémentaires** :

- **Niveau 1 — Sauvegarde SQL** : jobs AG-aware (sections 12.1 et 12.2). Sauvegarde les bases individuellement, dans S:\SQL\Backup\.
- **Niveau 2 — Sauvegarde VM-level** : sauvegarde complète des VMs SQLS01 et SQLS02, via **Veeam Backup & Replication** (ou un outil équivalent). Sauvegarde les VMs entières, dans un volume dédié sur l'hyperviseur ou sur un stockage réseau.

Les deux niveaux sont complémentaires : le niveau 1 permet une restauration granulaire (une base, une table), le niveau 2 permet une restauration complète (VM entière, OS, configuration).

12.3.1 Sauvegarde SQL (jobs AG-aware) — déjà couvert

- Cette approche est déjà couverte par les sections 12.1 et 12.2 :
- Jobs de sauvegarde FULL/DIFF/LOG sur les 13 bases + ASPState.
- Garde AG-aware (sys.fn_hadr_backup_is_preferred_replica).
- Sauvegardes stockées dans S:\SQL\Backup\ sur le nœud primaire.

12.3.2 Sauvegarde VM-level (Veeam Backup & Replication)

Veeam Backup & Replication est un outil de sauvegarde VM-level qui supporte nativement les Availability Groups SQL Server. Il est installé sur une **VM dédiée** (par exemple OPT-BKP-01) et se connecte à l'hyperviseur HYPER-V1 pour sauvegarder les VMs.

Principe :

- Veeam sauvegarde les VMs SQLS01 et SQLS02 en entier (OS, disques, configuration).
- Avec **Application-Aware Processing** activé, Veeam utilise VSS pour geler les écritures SQL Server avant le snapshot, garantissant une **cohérence applicative**.
- Pour les nœuds secondaires, Veeam crée automatiquement une sauvegarde en **Copy-Only**, qui ne casse pas la chaîne de log de l'AG.

Prérequis :

- Une VM dédiée pour Veeam (OPT-BKP-01), avec au moins 4 GB de RAM et 2 cœurs.
- Un **volume de sauvegarde dédié** sur l'hyperviseur ou sur un stockage réseau (par exemple \\OPT-FS03\VeeamBackups$).
- Veeam Backup & Replication Community Edition (gratuit jusqu'à 10 workloads).

Configuration recommandée :

- ✓ Installer Veeam Backup & Replication sur OPT-BKP-01.
- ✓ Ajouter l'hyperviseur HYPER-V1 comme source.
- ✓ Créer un **job de sauvegarde** pour SQLS01 et SQLS02 :
- **Mode** : Application-Aware Processing activé.
- **Type** : Copy-Only (pour ne pas interférer avec les jobs SQL AG-aware).
- **Fréquence** : quotidienne (par exemple à 23h00).
- **Rétention** : 7 jours, 4 semaines, 12 mois.

- ✓ Configurer un **volume de sauvegarde dédié** (différent du disque S:\).
- ✓ Tester la **restauration d'une base** depuis la sauvegarde VM-level.

Avantages :

- Sauvegarde complète (OS, applications, configuration).
- Restauration granulaire possible (une base depuis une sauvegarde VM).
- Cohérence applicative via VSS.
- Support natif des Availability Groups.

Inconvénients :

- Nécessite une VM dédiée et un volume de sauvegarde.
- Licence payante au-delà de 10 workloads (Community Edition).

RPO plus élevé que la sauvegarde SQL native (au mieux quotidien).

12.3.3 Comparison des deux approaches

Critère	Sauvegarde SQL (Natif)	Sauvegarde VM-level (Veeam)
Granularité	Base, table, enregistrement	VM entière, ou base individuelle (via Veeam Explorer)
RPO (Objectif de point de récupération)	Très faible (de l'ordre de la minute via les journaux de transactions)	Quotidien (ou selon la fréquence des snapshots)
RTO (Objectif de temps de récupération)	Faible (heures)	Moyen (heures à jours selon volumétrie)
Sauvegarde OS	Non	Oui
Sauvegarde configuration	Non	Oui (système, registres, services annexes)
Support Always On	Oui (AG-aware natif)	Oui (avec l'option Application-Aware Processing)
Coût	Inclus dans les licences SQL Server	Dépend de la licence Veeam (inclus dans l'édition standard/enterprise)

Critère	Sauvegarde SQL (Natif)	Sauvegarde VM-level (Veeam)
Complexité	Moyenne (scripts de maintenance, gestion des fichiers de backup)	Faible (centralisé et unifié au niveau de l'infrastructure virtuelle)

12.3.4 Recommandations pour le PoC et la production

Recommandation : **utiliser les deux approches en complément**.

- **Sauvegarde SQL** : pour la restauration granulaire et le RPO faible.
- **Sauvegarde VM-level** : pour la restauration complète en cas de sinistre.

Pour le PoC :

- Installer **Veeam Community Edition** sur une VM dédiée.
- Configurer un job de sauvegarde quotidien pour SQLS01 et SQLS02.
- Tester la restauration d'une base depuis la sauvegarde VM-level.
- **Ne pas utiliser les checkpoints Hyper-V** en routine (voir section 12.3.5).

Pour la production :

- **Veeam Backup & Replication** avec licence complète.
- **Volume de sauvegarde dédié** (différent du disque S:\).
- **Réplication hors site** (site secondaire ou cloud).
- **Tests de restauration réguliers** (mensuels).

13. Checklist complète des tests de bascule

- Basculement manuel planifié et automatique simulé.
- État Synchronized de toutes les bases après bascule ; failback vers le nœud d'origine.
- Coupure réseau simulée, test du witness, bascule de l'IP du Listener.
- Connexion des 13 applications, persistance ASPState, mesure du RTO réel.
- Jobs SQL Agent toujours fonctionnels sur le nouveau primaire (script 7.2.1).
- Certificat TDE présent et fonctionnel sur le nouveau primaire (script 7.1.1).

14. Sécurité réseau

- Port 5022 limité au trafic inter-nœuds SQLS01 ↔ SQLS02 (192.168.1.251/.252).
- Port 1433 réservé aux sous-réseaux IIS et OPT-RP-01.
- TLS 1.2 minimum, certificat SQL Server signé par une AC interne (AD CS).

15. Gestion des mises à jour SQL Server

1. Mettre à jour SQLS02 (secondaire) en premier.
2. Vérifier le retour à l'état Synchronized.
3. Basculer manuellement le rôle Primary vers SQLS02.
4. Mettre à jour SQLS01 (désormais secondaire).
5. Rebasculer vers SQLS01 si c'est le primaire nominal, puis valider (section 13).

16. Annexe — Liste des scripts fournis

Tous les scripts sont livrés dans Scripts_SQLServer2022_AlwaysOn_optimedit.zip (33 fichiers), nommés selon la numérotation des sections de ce guide.

Script	Nœud(s)	Étape
0.0.0.preflight_check.ps1	Poste admin (ou SQLS01)	0.1 Verification prealable (pre-vol)
3.0.0.manage_secrets.ps1	SQLS01+02	3.0 Gestion des secrets
3.1.0.create_storage_directories.ps1	SQLS01+02	3.1 Dossiers SQL (+ Audit)
3.2.0.create_sql_service_accounts.ps1	DC03	3.2 Comptes AD SQL
3.2.1.grant_service_logon_rights.ps1	SQLS01+02	3.2 Droits locaux
3.3.0.install_sql_server_2022-v2.ps1	SQLS01+02	3.3 Installation SQL
3.3.3.1.AddLocalAdminToSysadmin.ps1	SQLS01+02	3.3.3.1 Ajout admin local aux sysadmins
3.3.3.3.CheckPrivateKeyPermissions.ps1	SQLS01+02	3.3.3.3 Vérification permissions clé privée
3.3.3.5.CheckSqlCertificate.ps1	SQLS01+02	3.3.3.5 Vérification certificat SQL
3.3.3.6.RestartSqlCertificate.ps1	SQLS01+02	3.3.3.6 Redémarrage certificat SQL
3.4.0.configure_network_firewall.ps1	SQLS01+02	3.4 Pare-feu
4.1.0.install_failover_clustering_feature.ps1	SQLS01+02	4.1 Feature cluster
4.2.0.validate_cluster.ps1	SQLS01/02	4.2 Validation
4.3.0.create_cluster.ps1	SQLS01/02	4.3 Création cluster
4.4.0.configure_cluster_quorum.ps1	SQLS01/02	4.4 Quorum
5.1.0.enable_alwayson_feature.ps1	SQLS01+02	5.1 Activer AlwaysOn
5.2.0.create_hadr_endpoint.sql	SQLS01+02	5.2 Endpoint
5.3.0.create_availability_group.sql	SQLS01	5.3 Créer l'AG
5.4.0.join_availability_group_secondary.sql	SQLS02	5.4 Rejoindre l'AG
5.5.0.create_ag_listener.sql	SQLS01	5.5 Listener
5.6.0.test_failover.ps1	SQLS02	5.6 Test failover
6.1.0.create_application_databases.sql	SQLS01	6.1 Créer les 13 bases
6.2.0.add_databases_to_ag.sql	SQLS01	6.2 Ajout à l'AG
7.1.0.enable_tde.ps1	SQLS01	7.1 TDE
7.1.1.restore_tde_cert_on_secondary.ps1	SQLS02	7.1.1 Certificat TDE (correctif)
7.2.0.create_maintenance_jobs.sql	SQLS01	7.2 Jobs Agent
7.2.1.sync_maintenance_jobs_secondary.sql	SQLS02	7.2.1 Jobs sync (correctif)

7.3.0.create_alerts.sql	SQLS01+02	7.3 Alertes
7.4.0.enable_audit.sql	SQLS01+02	7.4 Audits
7.5.0.optimize_tempdb.sql	SQLS01+02	7.5 TempDB
8.1.0.create_iis_service_accounts.ps1	DC03	8.1 Comptes svc_*
8.2.0.create_GG_SVC_AppPools.ps1	DC03	8.2 Groupe AD IIS
8.3.0.create_aspstate_database.ps1	SQLS01	8.3 ASPState (avant login)
8.3.1.add_aspstate_to_ag.sql	SQLS01	8.3 Ajout ASPState à l'AG
8.4.0.create_sql_login_iis_group.sql	SQLS01	8.4 Login SQL (après ASPState)
8.5.0.configure_iis_sessionstate.ps1	Serveurs IIS	8.5 web.config
9.4.1 Création du partage SQLSecrets$	OPT-FS03	9.4.1 Création du partage SQLSecrets$
9.4.2.backup_secrets_to_fileserver.ps1	SQLS01+02	9.4.2 Sauvegarde des secrets vers OPT-FS03

17. Prochaines étapes possibles

- Guide de gestion des incidents Always On (NOT SYNCHRONIZING, endpoint bloqué, redo queue élevée).
- Pack de scripts de restauration complète de l'AG (disaster recovery).
- Migration vers Ola Hallengren **MaintenanceSolution.sql** pour la maintenance des 14 bases.
- Centraliser la gestion des secrets avec Azure Key Vault (ou HashiCorp Vault) pour remplacer le modèle actuel à clé AES locale par nœud, utile si le nombre de serveurs croît.
- Automatisation via Desired State Configuration (DSC) ou Ansible pour un déploiement reproductible (partie serveurs backends IIS déjà géré par Ansible).